Robotics Research Lab
Department of Computer Science
University of Kaiserslautern-Landau

Master Thesis

Learning-Based Online Adaptation for Control of a Hydraulic Manipulator

Kevin Kudakwashe Murera

Tuesday 21st July, 2026

Master Thesis

Learning-Based Online Adaptation for Control of a Hydraulic Manipulator

Robotics Research Lab
Department of Computer Science
University of Kaiserslautern-Landau

Kevin Kudakwashe Murera

Day of issue	:	TODO: date of issue
Day of release	:	TODO: date of submission

Reviewer	:	Prof. Dr. Karsten Berns
Supervisor	:	TODO: supervisor

Hereby I declare that I have self-dependently composed the Master Thesis at hand. The sources and additives used have been marked in the text and are exhaustively given in the bibliography.

Tuesday 21st July, 2026 – Kaiserslautern

(Kevin Kudakwashe Murera)

Acknowledgements

TODO: acknowledgements.

Abstract

TODO: abstract. Hydraulic manipulators are hard to control precisely because the actuator response is nonlinear, load dependent, and uncertain. This thesis investigates whether a hydraulic manipulator can achieve accurate and robust task-space tracking by combining a nominal Damped Least-Squares (DLS) controller, an upper-level residual PPO policy, learned hydraulic actuator dynamics, and online latent adaptation through an Arm Adaptation Module (AAM).

Contents

1 Introduction **1**

1.1 Motivation and Application Context 1
1.2 Problem Statement 1
1.3 Research Questions 1
1.4 Objectives 1
1.5 Contributions 1
1.6 Scope and Limitations 1
1.7 Thesis Structure 1

2 Fundamentals and Related Work **3**

2.1 Manipulator Actuation and Control Fundamentals 3
2.1.1 Application context: implement carriers with hydraulic booms . . . 3
2.1.2 The electro-hydraulic actuation chain 4
2.1.3 High-fidelity simulation as data source, not training plant 6
2.2 Kinematics and Task-Space Tracking 6
2.2.1 Differential kinematics 6
2.2.2 From joints to cylinders 6
2.3 Model-Based and Conventional Control Approaches 7
2.4 Data-Driven System Identification and Learned Dynamics 8
2.4.1 Learned forward and inverse dynamics 8
2.4.2 The closed-loop identifiability trap 8
2.5 Reduced-Order Modeling 9
2.6 Domain Randomization and Latent Adaptation 10
2.6.1 Robustness through randomized training 10
2.6.2 From robustness to adaptation 10
2.7 Reinforcement Learning and Residual Control 11
2.7.1 Policy optimization 11
2.7.2 Residual policies over a nominal controller 12
2.7.3 Reinforcement learning on hydraulic machines 12
2.8 Research Gap and Thesis Positioning 13

3 System Fundamentals and Problem Formulation 15
3.1 Manipulator Platform . . . 15
3.2 Kinematic Structure and Coordinate Mappings . . . 17
3.2.1 Joint space and cylinder space . . . 17
3.2.2 Task space and the calibrated cutter Jacobian . . . 17
3.3 Actuation and Measured State . . . 18
3.4 High-Fidelity Simscape Model . . . 19
3.4.1 Mechanical subsystem . . . 19
3.4.2 Hydraulic subsystem . . . 20
3.4.3 What the model is validated to reproduce . . . 21
3.5 IsaacLab Environment . . . 23
3.6 Hierarchical Control Architecture . . . 24
3.7 Problem Formulation . . . 25
3.8 Evaluation Criteria at System Level . . . 25

4 Data Generation and Preparation 27
4.1 Dataset Objectives and Design Requirements . . . 27
4.2 Randomized Parameter Space . . . 29
4.2.1 Sampling design . . . 29
4.2.2 Hydraulic parameters . . . 29
4.2.3 Compliance-equivalent representation of hydraulic compressibility . 29
4.2.4 Mechanical parameters . . . 31
4.3 Closed-Loop Data Collection . . . 32
4.3.1 The controller as a data-collection instrument . . . 33
4.3.2 Trajectories . . . 33
4.3.3 Execution and logging . . . 34
4.4 Open-Loop Excitation Experiments . . . 35
4.4.1 Why the closed-loop batch was not enough . . . 35
4.4.2 Design of the excitation signal . . . 35
4.4.3 Plant draws and acceptance . . . 36
4.5 Combining Closed-Loop and Open-Loop Datasets . . . 37
4.6 Feature and Target Construction . . . 38
4.7 Preprocessing and Data Splits . . . 39
4.8 Dataset Diagnostics . . . 39

5 Learned Models and Online Adaptation 41
5.1 Modeling Overview and Common Notation . . . 41
5.1.1 Cylinder-space signals . . . 41
5.1.2 Two actuator representations, one contract . . . 42
5.1.3 Common conventions . . . 42
5.1.4 Data provenance in brief . . . 44
5.2 Inverse Dynamics Model . . . 44
5.2.1 Input–output contract . . . 44
5.2.2 Architecture and supervised training . . . 45
5.2.3 Paired control fine-tuning . . . 46

5.3 Forward Dynamics Model . . . 46
5.3.1 Input–output contract and supervised baseline . . . 46
5.3.2 Why the supervised score was not believed . . . 47
5.3.3 Strict open-loop excitation dataset . . . 47
5.3.4 Strict retraining objective . . . 49
5.3.5 Validation: valve authority restored . . . 49
5.4 Reduced-Order Model . . . 50
5.4.1 Motivation and design position . . . 50
5.4.2 Input–output contract . . . 51
5.4.3 Grey-box structure . . . 51
5.4.4 Dataset and training . . . 52
5.4.5 Offline validation . . . 53
5.5 Latent Representation of Randomized Dynamics . . . 54
5.6 Arm Adaptation Module . . . 54
5.7 Training and Model-Selection Protocol . . . 54
5.8 Offline Model Evaluation . . . 54

6 Control Framework and Simulation Integration 55
6.1 Nominal Task-Space Controller . . . 55
6.2 Residual PPO Formulation . . . 55
6.3 PPO1-A: Nominal ID-FD Pipeline . . . 55
6.4 PPO1-B: Randomized/Robust ID-FD Pipeline . . . 55
6.5 PPO2-A: Nominal ROM Pipeline . . . 55
6.6 PPO2-B: Randomized/Robust ROM Pipeline . . . 55
6.7 PPO2-C: AAM-Conditioned ROM Pipeline . . . 55
6.8 Optional Oracle-Latent Diagnostic . . . 56
6.9 IsaacLab Integration and Timing . . . 56
6.10 Stability and Constraint Handling . . . 56

7 Experimental Evaluation and Discussion 57
7.1 Evaluation Questions and Hypotheses . . . 57
7.2 Experimental Matrix and Fairness Controls . . . 57
7.3 Model-Level Experiments . . . 57
7.4 PPO Training Behaviour . . . 57
7.5 Task-Space Trajectory Tracking . . . 57
7.6 Robustness and Generalization Tests . . . 57
7.7 Adaptation Experiments . . . 57
7.8 Ablation Studies . . . 58
7.9 Metrics and Statistical Reporting . . . 58
7.10 Integrated Discussion . . . 58
7.11 Threats to Validity . . . 58

8 Conclusion and Outlook **59**
8.1 Summary of the Approach 59
8.2 Answers to the Research Questions 59
8.3 Main Contributions 59
8.4 Limitations 59
8.5 Future Work 59

A System Parameters and Coordinate Mappings **61**

B Dataset and Randomization Details **63**

C Model Architectures and Training Settings **65**

D PPO and IsaacLab Configuration **67**

E Supplementary Results **69**

Bibliography **71**

1. Introduction

1.1 Motivation and Application Context

[TODO: write motivation]

1.2 Problem Statement

[TODO: write problem statement]

1.3 Research Questions

[TODO: state research questions RQ1–RQ4]

1.4 Objectives

[TODO: state objectives]

1.5 Contributions

[TODO: list contributions]

1.6 Scope and Limitations

[TODO: define scope and limitations]

1.7 Thesis Structure

[TODO: summarize chapters 2–8]

2. Fundamentals and Related Work

Chapter 1 argued that precise task-space tracking with a hydraulic mowing arm is limited less by kinematics than by the actuator. The path from a valve command to actual cylinder motion is nonlinear and load dependent, and it differs from one machine, season, or attachment to the next. This chapter collects the background needed to judge that argument and to understand the design decisions made later in the thesis. Section 2.1 introduces the machine class and its electro-hydraulic actuation. Section 2.2 recalls the kinematic tools behind the nominal controller, and Section 2.3 reviews how conventional model-based control deals with hydraulic uncertainty. The following sections turn to the learning methods this work builds on: data-driven dynamics models and the pitfalls of identifying them from closed-loop data (Section 2.4), reduced-order and grey-box modeling (Section 2.5), domain randomization and latent adaptation (Section 2.6), and residual reinforcement learning (Section 2.7). Section 2.8 then states the gap that remains and how the PPO1 and PPO2 pipelines of this thesis address it. Only material that is needed for the later chapters is included; the concrete system definition and problem formulation follow in Chapter 3.

2.1 Manipulator Actuation and Control Fundamentals

2.1.1 Application context: implement carriers with hydraulic booms

The machine studied in this thesis belongs to a class that is easy to overlook because it is so common: hydraulic booms mounted on road-going implement carriers, doing repetitive maintenance work along traffic infrastructure. Figure 2.1 shows the concrete instance, the MULAG FME 500 front mower on a Unimog carrier [MULAG Fahrzeugwerk, Heinz Wössner GmbH u. C
A multi-link boom carries a 1.2 m flail mowing head to the roadside verge, reaches up to about 6.5 m to either side of the vehicle, and must hold the head in a shallow band above the ground while the vehicle creeps along the road. The working envelope in Figure 2.1(b) makes the difficulty concrete. The same arm that mows a flat verge beside the wheels



(a) (b)

Figure 2.1: The plant considered in this thesis. (a) MULAG FME 500 front mower on a Unimog implement carrier during roadside mowing; the multi-link hydraulic boom holds the flail head against the verge while the vehicle drives. (b) Manufacturer's working envelope of the boom, spanning roughly 8 m of height and reaching below road level for embankment mowing. Images: MULAG Fahrzeugwerk [MULAG Fahrzeugwerk, Heinz Wössner GmbH u. Co. KG 26].

must also reach over guardrails, down embankments to about 3 m below road level, and up to 8 m for hedge work, with the same linkage and the same operator. In practice, head height, tilt, and lateral position are adjusted continuously by a human through open-loop joystick commands. It is skilled and fatiguing work, done while also driving the vehicle. Automating the tracking part of this task is the practical motivation of this thesis.

Hydraulic actuation dominates this machine class for reasons that remain valid despite the rise of electric drives. Cylinders deliver very high force from small, rugged, and inexpensive components; they tolerate shock loads such as the mowing head striking a curbstone; and the carrier vehicle already provides hydraulic power through its power take-off. The price is paid in controllability. An electric joint drive responds to current within milliseconds and nearly linearly. A hydraulic cylinder responds through a chain of nonlinear energy conversions whose effective gain can change by an order of magnitude across the workspace. Mattila et al. [Mattila 17] survey two decades of research on this tension between the force density of hydraulic manipulators and the difficulty of controlling them precisely.

2.1.2 The electro-hydraulic actuation chain

Figure 2.2 traces the signal path that every command to the arm must traverse. A normalized electrical command displaces the spool of a proportional valve, which opens metering orifices toward the supply and tank lines. To first approximation, the resulting flow follows the orifice law [Merritt 67]

$$q \;=\; c_d\, w\, x_v \sqrt{\tfrac{2}{\rho}\,\bigl|p_s - p_{\mathrm{load}}\bigr|}\;\operatorname{sgn}\bigl(p_s - p_{\mathrm{load}}\bigr), \tag{2.1}$$

with discharge coefficient c_d, orifice area gradient w, spool position x_v, oil density ρ, supply pressure p_s, and load pressure p_{load}. Two properties of (2.1) shape everything that follows

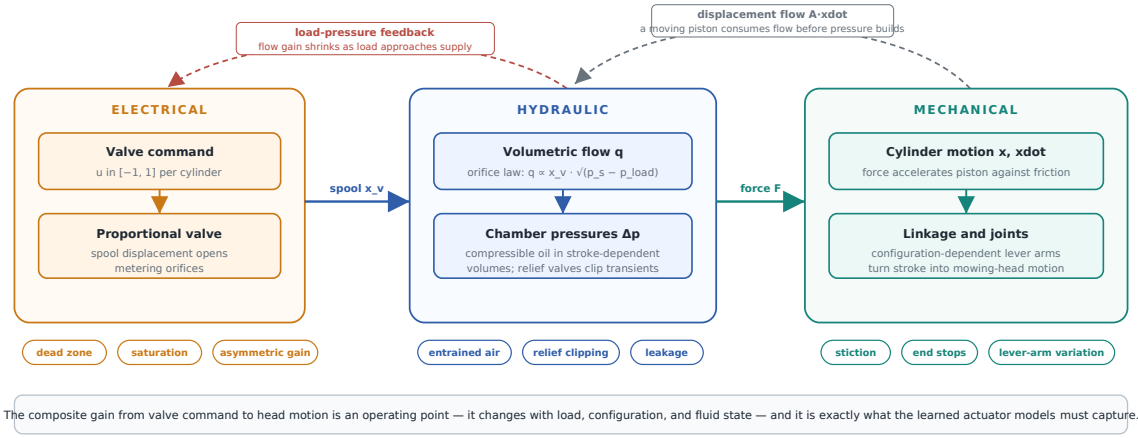


Figure 2.2: The electro-hydraulic actuation chain from valve command to mowing-head motion, grouped by energy domain. The tags beneath each domain name its dominant nonlinearities. The two dashed return paths are the physical feedback loops that make the effective gain of the chain an operating point rather than a constant: the load pressure throttles the valve flow through the square-root term of (2.1), and the moving piston itself consumes flow before pressure can build.

in this thesis. First, the flow gain depends on the square root of the pressure margin. The same spool displacement therefore moves the cylinder quickly under light load and barely at all when the load pressure approaches supply; the effective valve authority is an operating point, not a constant. Second, the relation is directional. Extending against gravity and retracting with it see different pressure margins and hence different gains. Around the spool null, machining overlap produces a dead zone in which small commands move nothing at all, and at the other end of the stroke the spool saturates [Jelali 03].

The delivered flow does not translate directly into motion either. Oil is compressible, the more so when air is entrained, so each cylinder chamber acts as a nonlinear spring. Its stiffness scales with the effective bulk modulus and inversely with the chamber volume, and the volume itself changes with piston position. Pressure relief valves clip the dynamics whenever a load spike or an aggressive command drives a chamber to its limit, which happens routinely in mowing when the head rests on the ground. Internal leakage bleeds pressure between chambers, seal friction opposes motion, and the load pressure feeds back into the valve flow through (2.1). Taken together, this is the plant that the learned models of this thesis must represent: several interacting nonlinearities, some of them non-smooth (relief, dead zone, stiction), and parameters such as bulk modulus, leakage, and friction that drift with temperature and wear.

One structural point matters for the control architecture. The chain in Figure 2.2 ends in the mechanical linkage, where cylinder translation becomes joint rotation through lever arms that vary with configuration. Uncertainty entering at the valve is therefore filtered, amplified, and coupled by the mechanism before it appears as head-position error. This is why the models of Chapter 5 are formulated in *cylinder space*, directly across the hydraulic conversion stages, rather than in task space where hydraulic and kinematic effects blur together.

2.1.3 High-fidelity simulation as data source, not training plant

The hydraulic subsystems above can be modeled in detail in component-level simulation environments such as Simscape [The MathWorks, Inc. 24], and Chapter 4 builds on such a model of the FME 500 arm. It is tempting to close the loop and train a reinforcement-learning policy directly against this model, but the computational profiles do not match. The stiff pressure dynamics force small integration steps, and the solver processes one machine instance at a time, while modern policy optimization spreads its sample cost over thousands of plants simulated in parallel on a GPU [Makoviychuk 21]. This thesis resolves the mismatch by separation: the high-fidelity model is used offline as a data generator, and compact learned representations of the actuator carry its behavior into the parallel training environment. How this workflow fits together is shown later, in Figure 2.3, once its ingredients have been introduced.

2.2 Kinematics and Task-Space Tracking

2.2.1 Differential kinematics

The mowing task is specified where the work happens, as a reference path for the cutter in the coordinates of the vehicle. Task-space control therefore rests on the differential kinematics of the arm [Siciliano 09]. Writing $\boldsymbol{q}$ for the joint configuration and $\boldsymbol{r}(\boldsymbol{q})$ for the forward-kinematic position of the tool point, small motions are related through the manipulator Jacobian $\boldsymbol{J}(\boldsymbol{q}) = \partial \boldsymbol{r} / \partial \boldsymbol{q}$,

$$\dot{\boldsymbol{r}} = \boldsymbol{J}(\boldsymbol{q})\, \dot{\boldsymbol{q}}. \tag{2.2}$$

Inverting (2.2) to obtain joint or cylinder commands from a desired tool velocity is the core of resolved-rate control. The plain pseudoinverse solution is fragile. Near singular configurations, which the mowing arm visits routinely (for instance with the boom almost fully folded at the near edge of the envelope in Figure 2.1(b)), small task-space errors demand extreme joint rates. The standard remedy, adopted unchanged in this thesis, is the damped least-squares (DLS) inverse proposed independently by Wampler [Wampler 86] and by Nakamura and Hanafusa [Nakamura 86],

$$\dot{\boldsymbol{q}} = \boldsymbol{J}^{\top}\Big(\boldsymbol{J}\boldsymbol{J}^{\top} + \lambda^{2}\boldsymbol{I}\Big)^{-1}\dot{\boldsymbol{r}}^{*}, \tag{2.3}$$

which trades a bounded amount of tracking accuracy for bounded commands through the damping factor λ. DLS is conservative, well understood, and cheap. These are the properties one wants in the layer of the controller that must never misbehave.

2.2.2 From joints to cylinders

Equations (2.2) and (2.3) are textbook material for revolute arms. The complication on a hydraulic boom is that the actuated coordinates are not the joint angles but the cylinder lengths. Each cylinder closes a triangle over its joint, so the cylinder-to-joint mapping is a configuration-dependent, monotone but nonlinear relation, and its derivative, the lever arm, varies over the stroke. Two practical consequences follow. First, the Jacobian actually used for control is the composition of the task Jacobian with the joint-to-cylinder map, so that

DLS can be solved directly for cylinder-velocity commands. This thesis uses a calibrated 3×4 cutter-to-cylinder Jacobian identified on the simulated plant (Chapter 3). Second, the mapping couples uncertainty: an error in one cylinder's realized motion appears in the task error of the whole arm, weighted by lever arms that change along the path. The FME 500 additionally has a passive fifth joint that is mechanically coupled to the head, which constrains which task directions can be controlled independently; Chapter 3 works out the consequences.

The kinematic layer, however accurate, contains an assumption that this thesis deliberately does not make: that a commanded cylinder velocity is a realized cylinder velocity. Everything in Section 2.1.2 argues against that assumption on this actuator class. The kinematic controller is therefore treated as a *nominal* structure whose commands are realized, or not, by the hydraulic layer. The learned models of this thesis live in that realization gap.

2.3 Model-Based and Conventional Control Approaches

Industrial practice on machines like the FME 500 is open-loop operator control: the joystick maps to valve current and the human closes every loop. The control literature offers a spectrum of alternatives, and it is worth being clear about why none of them alone settles the problem this thesis addresses.

Classical cascade structures, an outer position loop commanding an inner velocity or pressure loop with fixed PI or PD gains, are the workhorse of hydraulic servo control [Jelali 03]. They are simple and robust, but fixed gains must be tuned for the worst case across the operating envelope, and Section 2.1.2 showed that the plant gain varies by an order of magnitude across that envelope. The result is a controller that is sluggish where the plant is weak or oscillatory where it is strong. On a boom whose lever arms change with every meter of reach, one gain set cannot be right everywhere.

Model-based nonlinear methods attack the problem directly. Computed-torque control cancels the rigid-body dynamics if they are known [Slotine 91]; backstepping designs extend the idea through the pressure dynamics; adaptive versions estimate uncertain parameters online [Åström 95]; robust versions bound their effect. Applied to hydraulic manipulators, these designs achieve impressive tracking in laboratory settings. Their guarantees, however, are conditioned on structural model knowledge: valve flow curves, friction models, inertia parameters. Obtaining and maintaining that knowledge for a commercial implement, produced in variants and operated across seasons, is expensive, and Mattila et al. [Mattila 17] conclude their survey with essentially this observation: the accuracy of model-based hydraulic control is bounded by the accuracy of the models, and the models are the bottleneck.

This thesis draws a specific lesson from that literature rather than discarding it. Structured controllers are kept where structure is reliable. The kinematic DLS layer of Section 2.2 is retained as a safe, interpretable nominal controller, because differential kinematics *is* accurately known. Learning is directed at the part where first-principles modeling is the bottleneck, the mapping between cylinder-space commands and realized cylinder motion, and, in a second layer, at residual corrections on top of the nominal controller (Section 2.7). The architecture is hybrid by design, not black-box.

2.4 Data-Driven System Identification and Learned Dynamics

2.4.1 Learned forward and inverse dynamics

Where deriving a model is the bottleneck, one can measure it instead. In robot model learning [Nguyen-Tuong 11], two complementary mappings are distinguished. A *forward dynamics* model predicts the next state, or state increment, from the current state and the applied command,

$$\hat{\boldsymbol{s}}_{k+1} = \boldsymbol{s}_k + f_\theta(\boldsymbol{s}_k, \boldsymbol{u}_k), \tag{2.4}$$

and an *inverse dynamics* model predicts the command that realizes a desired transition, $\boldsymbol{u}_k = g_\phi(\boldsymbol{s}_k, \boldsymbol{s}^*_{k+1})$. Neural networks are the standard function class when the underlying physics is non-smooth and history dependent, as it is here. For control, one-step accuracy of (2.4) is necessary but not sufficient. A model used inside a planner or a training loop is iterated, feeding its own output back as input,

$$\hat{\boldsymbol{s}}_{k+H} = \hat{\boldsymbol{s}}_{k+H-1} + f_\theta\Big(\hat{\boldsymbol{s}}_{k+H-1}, \boldsymbol{u}_{k+H-1}\Big), \qquad \hat{\boldsymbol{s}}_k = \boldsymbol{s}_k, \tag{2.5}$$

so small one-step errors compound over the horizon H, and a model selected purely by one-step loss can diverge within a few hundred milliseconds of recursive prediction [Nagabandi 18]. Multi-step rollout objectives, which apply (2.5) during training and penalize the accumulated error, counteract this compounding directly and are used for every dynamic model trained in this thesis. The probabilistic-ensemble line of work [Chua 18] addresses the same problem by additionally tracking how model uncertainty grows along the rollout. Where a single sample cannot summarize the relevant past, for instance pressure transients and valve history, recurrent architectures [Hochreiter 97, Cho 14] extend (2.4) with an internal state that integrates a window of history. The reduced-order model of Chapter 5 takes this form.

2.4.2 The closed-loop identifiability trap

The dominant practical danger in learning (2.4) is not underfitting but learning the wrong thing well. System-identification theory has long formalized when data determines a plant: informative experiments require inputs that excite the dynamics independently of the feedback law [Ljung 99, Isermann 11]. Data collected in closed loop violates this in a treacherous way. The controller computes the command from the state history, so command and state are strongly correlated, and a flexible model can reach excellent prediction scores by exploiting the state history alone. It then behaves like a sophisticated persistence model that ignores the input channel. The failure is silent, because validation data from the same closed-loop distribution keeps rewarding the shortcut. Schoukens and Ljung [Schoukens 19] emphasize that for nonlinear systems the design of informative excitation is, if anything, more critical than for linear ones.

This is not a hypothetical concern in this thesis; it is a measured event. The first forward-dynamics model of Chapter 5, trained on closed-loop operation data, reached one-step R^2 near 0.999 while an input ablation showed that it had learned almost no dependence on the valve command at all. The open-loop excitation campaign of Chapter 4, valve sequences applied directly to the plant without any feedback controller in the loop, exists

specifically to break this correlation and make the causal valve-to-motion path identifiable. The methodological rule that follows is applied to every learned model in this work: input-ablation tests with zeroed and shuffled commands are part of model acceptance, and a supervised score is never accepted as evidence of control authority on its own.

2.5 Reduced-Order Modeling

The term *reduced-order model* comes from a large literature on projecting high-dimensional dynamical systems onto low-dimensional subspaces that preserve their dominant behavior; Benner et al. [Benner 15] survey this family, including parametric variants that remain valid across a family of plants. The motivation is the one that matters here: a model cheap enough to evaluate thousands of times per second, yet faithful enough that conclusions drawn from it transfer to the full system. This thesis uses the term in the same spirit but with a learned construction. The full Simscape hydraulic model, with its many internal states, is reduced to a compact per-cylinder state (extension, extension velocity, pressure difference) whose transition is learned from data.

Between white-box projection and black-box regression lies the grey-box middle ground this thesis occupies. Physics-informed learning embeds known structure, such as conservation laws or integrator chains, into the hypothesis class, so that data is spent on what is genuinely unknown rather than on rediscovering known physics [Karniadakis 21]. Lutter et al. [Lutter 19] demonstrated the benefit for rigid-body dynamics by building Lagrangian structure into a network. The reduced-order model of Chapter 5 applies the same idea to hydraulics: the orifice law (2.1), the pressure-buildup equation, and the velocity–position integrator chain are explicit layers with trainable effective parameters, while neural components supply only the valve realization, the configuration-dependent load, and small bounded residuals. Two properties motivated this choice over a free-form sequence model. The trained effective parameters can be checked against physical plausibility, and the bounded residuals mean that even an off-distribution input cannot produce an unphysical state jump. A policy-training loop, which pushes its model off-distribution by design, benefits from both.

The closest precedent for embedding such a surrogate in policy training comes from legged robotics. Hwangbo et al. [Hwangbo 19] identified the rigid-body mechanics of their robot analytically, learned the response of its series-elastic actuators from measured data, trained a policy against the resulting hybrid simulation, and deployed it directly on the machine; later work industrialized this recipe in GPU-parallel simulators [Rudin 22]. Figure 2.3 shows how this thesis adopts the same division of labor with the roles recast for hydraulics. The PhysX articulation in IsaacLab plays the analytic rigid-body part, and the learned part is not a torque filter for a well-behaved electric drive but the entire hydraulic chain of Figure 2.2. That difference is why the surrogate here must be hydraulic-aware, structured around pressure dynamics and valve nonlinearity, and command-conditioned, so that it realizes desired cylinder commands on the plant side rather than filtering torques.

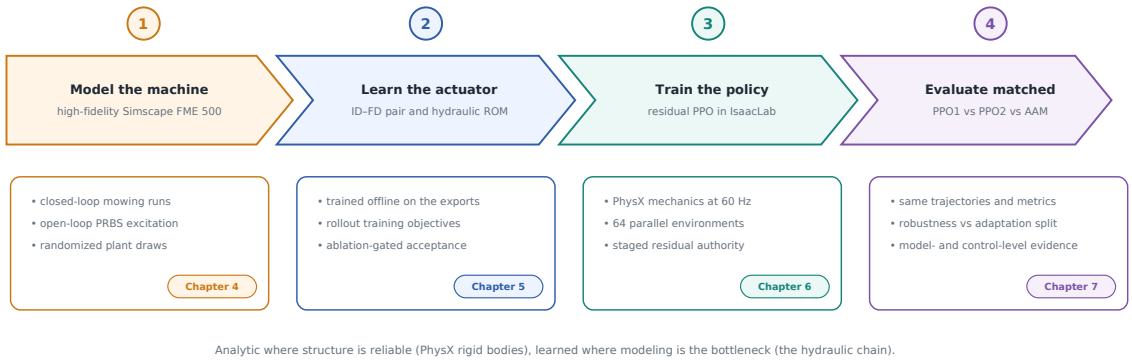


Figure 2.3: The four-step workflow of this thesis, following the division of labor introduced for legged robots by Hwangbo et al. [Hwangbo 19]: model what is reliably known, learn the actuator from data, train the policy against the combination, and evaluate under matched conditions. The cards list what each step contributes and the chapter that develops it.

2.6 Domain Randomization and Latent Adaptation

2.6.1 Robustness through randomized training

A model or policy trained on one simulated plant inherits every parameter of that plant as an implicit assumption. Domain randomization attacks this brittleness by training across a distribution of plants, originally over visual appearance for perception [Tobin 17], soon after over dynamics parameters such as masses, frictions, gains, and latencies for control [Peng 18]. The now-standard reading of those results is that a policy or model that must succeed on many plants converges to behavior that is *robust* across the family. For the hydraulic arm the corresponding family is physically meaningful rather than synthetic: effective bulk modulus varies with entrained air and temperature, valve gains and leakage drift with wear, and payload changes with the mounted attachment. Chapter 4 defines the randomized parameter space along these same axes.

2.6.2 From robustness to adaptation

Robustness, however, is a compromise by definition: one response that is acceptable everywhere is optimal nowhere. The alternative is to let the controller or model identify which member of the family it is currently facing and condition on that estimate (Figure 2.4). The teacher–student construction makes this trainable. During simulation the true randomized parameters $\boldsymbol{\xi}$ are known, and a teacher encoder compresses them, or their dynamic consequences, into a low-dimensional latent $\boldsymbol{z}$. A student module then learns to regress $\boldsymbol{z}$ from signals that exist outside simulation, namely the recent history of states, commands, and responses. Lee et al. [Lee 20] used this scheme to walk over terrain whose properties the robot cannot observe directly. Kumar et al. [Kumar 21] sharpened it into rapid motor adaptation: a first phase trains the policy together with an encoder of the privileged environment parameters, a second phase trains an adaptation module to recover the same latent from the observable state–action history, and at deployment only the history-based estimate is used, so the system adapts within fractions of a second

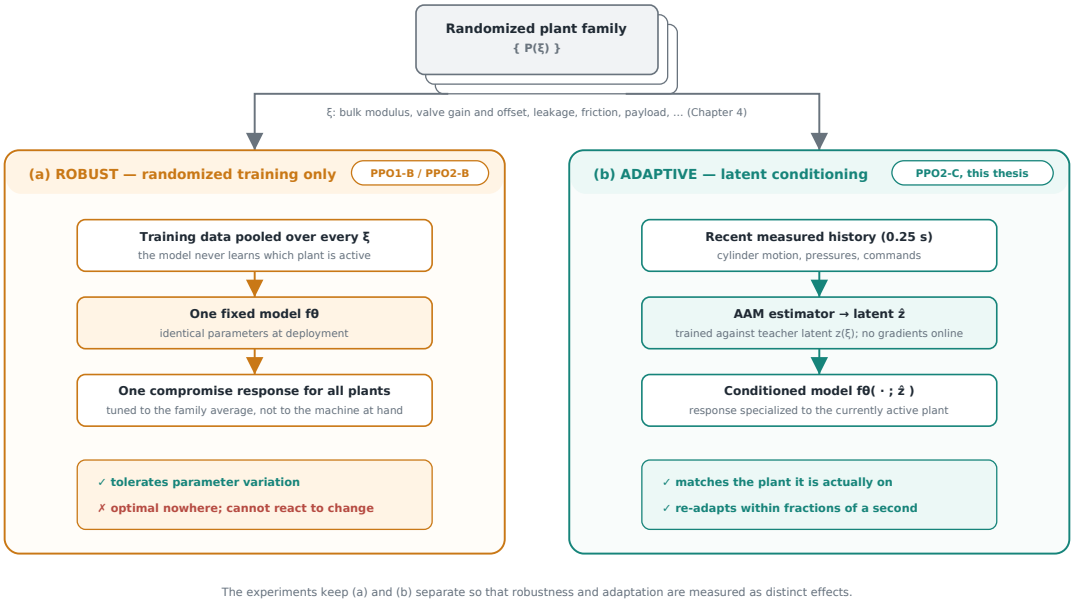


Figure 2.4: The distinction this thesis is built to measure, as two treatments of the same randomized plant family. (a) Randomized training alone produces a *robust* model: one fixed response that works acceptably everywhere and optimally nowhere. (b) Latent conditioning adds *adaptation*: an estimator infers a compact representation $\hat{\boldsymbol{z}}$ of the currently active plant from recent measured history, and the model changes its response accordingly. The estimator is trained against a teacher latent computed from the true parameters $\boldsymbol{\xi}$, but deploys on measurements alone.

without any gradient updates. Classical adaptive control pursues the same goal through online parameter estimation [Åström 95]; the learned variant replaces explicit parameter estimates with a task-shaped latent, trading physical interpretability for the ability to summarize many interacting parameters at once.

The Arm Adaptation Module (AAM) of this thesis (Chapter 5) is this construction applied to hydraulic actuation, following the adaptive path of Figure 2.4(b). The latent describes the currently experienced hydraulic and mechanical condition of the arm, the estimator reads it from a quarter second of cylinder-signal history, and the reduced-order model is conditioned on it. The experiments of Chapter 7 keep the robust and adaptive variants separate on purpose. PPO2-B, the randomized ROM without latent input, isolates what randomization alone buys; PPO2-C, with AAM conditioning, isolates what online adaptation adds. Collapsing the two into one comparison is a common confound in the sim-to-real literature, and this thesis is deliberately built to avoid it.

2.7 Reinforcement Learning and Residual Control

2.7.1 Policy optimization

Reinforcement learning frames control as sequential decision making: a policy $\pi_\vartheta(\boldsymbol{a} \mid \boldsymbol{o})$ maps observations to actions and is optimized to maximize expected cumulated reward [Sutton 18]. Among policy-gradient methods, proximal policy optimization (PPO) [Schulman 17]

has become the default in robot learning for its simplicity and stable large-batch behavior. It constrains each update through the clipped surrogate

$$\mathcal{L}^{\mathrm{CLIP}}(\vartheta) = \mathbb{E}_k\Big[\min\Big(\varrho_k(\vartheta)\,\hat{A}_k,\ \mathrm{clip}\Big(\varrho_k(\vartheta),\,1-\epsilon,\,1+\epsilon\Big)\hat{A}_k\Big)\Big], \tag{2.6}$$

where ϱ_k is the probability ratio between the new and old policy and $\hat{A}_k$ the estimated advantage, computed in practice with generalized advantage estimation [Schulman 16]. What makes on-policy methods like PPO practical for this thesis is throughput. GPU-parallel simulators run thousands of environment instances at once [Makoviychuk 21], and locomotion policies have been trained this way in minutes [Rudin 22]. This is the regime in which the full hydraulic model cannot run (Section 2.1.3) and the learned actuator surrogates can.

2.7.2 Residual policies over a nominal controller

Applying RL to the mowing arm from scratch would discard the reliable kinematic structure of Section 2.2 and force the policy to rediscover it from reward, slowly and with excursions that a heavy boom cannot afford. Residual reinforcement learning instead superimposes the policy on a hand-designed controller [Johannink 19, Silver 18],

$$\boldsymbol{u} = \boldsymbol{u}_{\mathrm{nom}}(\boldsymbol{o}) + \pi_\vartheta(\boldsymbol{o}), \qquad \|\pi_\vartheta\| \le \bar{a}, \tag{2.7}$$

so the nominal controller provides competent behavior from the first step, exploration happens in a small bounded band $\bar{a}$ around it, and the policy spends its capacity on the errors the nominal design cannot remove, which here means the hydraulic realization gap. Both pipelines of this thesis use this pattern: the DLS controller issues the nominal cylinder command and PPO contributes a bounded correction, with the residual authority widened in stages during training (Chapter 6).

2.7.3 Reinforcement learning on hydraulic machines

Applications of deep RL to hydraulically actuated arms are still few, and the closest of them sharpen rather than close the gap this thesis addresses. Egli and Hutter demonstrated RL-based trajectory tracking for the arm of a full-size hydraulic excavator, first in simulation-to-machine transfer experiments [Egli 20] and then as a general automation approach deployed on a walking excavator [Egli 22]. Their key ingredient is shared with this work: a data-driven model of the machine's actuator response, learned from recorded operation and used as the training plant for the policy. The differences define where this thesis stands. Their policy replaces the classical controller outright rather than correcting a structured nominal layer. Their actuator model is identified for one physical machine, so robustness across a plant family and online adaptation to a changing plant are outside its scope. And their evaluation compares against human operators and classical baselines rather than isolating the contribution of the actuator representation itself. The comparison that Chapter 7 is built around, same policy, same controller, different actuator models, has to the author's knowledge no published counterpart for hydraulic manipulators.

One property of residual RL cuts both ways and shapes the evaluation methodology of this thesis. Because the policy can compensate for a mediocre actuator model, closed-loop

Table 2.1: Positioning of this thesis against the reviewed lines of work. Columns: (I) learned model of *hydraulic* actuator dynamics; (II) training data audited for causal input identifiability; (III) online latent adaptation to a changing plant; (IV) structured nominal controller retained, with learning as a bounded residual; (V) matched comparison isolating the actuator representation. (✓) = partially or implicitly addressed.

Line of work	I	II	III	IV	V
Model-based hydraulic control [Mattila 17, Jelali 03]	—	(✓)	(✓)	✓	—
Actuator networks for legged robots [Hwangbo 19, Rudin 22]	—	✓	—	—	—
RL for hydraulic excavators [Egli 20, Egli 22]	✓	(✓)	—	—	—
Locomotion adaptation [Lee 20, Kumar 21]	—	—	✓	—	—
Residual reinforcement learning [Johannink 19, Silver 18]	—	—	—	✓	—
This thesis (PPO1 / PPO2 / AAM)	✓	✓	✓	✓	✓

tracking error alone cannot distinguish a good model from a good patch over a bad one. The protocol of Chapter 7 therefore reports model-level rollout metrics and the magnitude and smoothness of the residual actions alongside tracking error. A genuinely better actuator representation must show up as less residual correction, not merely as equal tracking.

2.8 Research Gap and Thesis Positioning

Laid side by side, the threads of this chapter each solve part of the problem and leave a specific remainder (Table 2.1 names the works). Hydraulic control theory provides structure and explains why fixed models fail, but its precise methods presuppose plant knowledge that a commercial mowing implement does not come with. Learned dynamics models replace derivation with data, but the identifiability literature, and the measured failure recounted in Section 2.4.2, shows that closed-loop operation data silently produces models without control authority. Learned actuator surrogates embedded in parallel simulation are proven for electric drives but not for actuators dominated by pressure dynamics, relief saturation, and valve dead zones. Domain randomization yields robustness, and the teacher–student construction yields fast online adaptation, but these results live almost entirely in locomotion, where the adapted quantity is the contact with the world rather than the actuator itself. RL has reached full-size hydraulic arms, but without a structured nominal layer, without adaptation, and without isolating the actuator representation. Residual RL integrates learning with a structured controller, yet published evaluations rarely control for the fact that the policy can mask actuator-model deficiencies.

What is missing is the combination, summarized in Table 2.1: cylinder-space actuator dynamics learned from deliberately identifiable data drawn from a high-fidelity hydraulic model; a compact, physics-structured, command-conditioned reduced model of those dynamics that can run in massively parallel policy training; an online latent-adaptation mechanism that conditions this model on the currently experienced plant; and a residual task-space control framework in which adaptive and non-adaptive actuator representations are compared under matched conditions, with model-level and control-level evidence kept separate. Assembling and evaluating exactly this combination on the MULAG FME 500 mowing arm is the contribution of this thesis. The PPO1 pipeline, the learned ID–FD

actuator pair under residual PPO, supplies the strongest non-adaptive learned baseline. The PPO2 pipeline, built on the hydraulic reduced-order model, carries the proposed method. The ablation ladder between them, nominal, randomized, and AAM-conditioned, is constructed so that robustness and adaptation are measured as separate effects rather than conflated ones.

The next chapter grounds this program in the concrete system: the simulated FME 500 plant, its coordinate spaces and measured signals, the hierarchical control architecture, and the formal statement of the tracking, robustness, and adaptation objectives.

3. System Fundamentals and Problem Formulation

Chapter 2 closed with a program: learn the hydraulic actuation of the FME 500 from identifiable data, wrap the learned models in a structured controller, and measure robustness and adaptation as separate effects. Before any of that can be made concrete, the system itself has to be pinned down. This chapter describes the manipulator and its two simulated representations exactly once, so that every later chapter can refer back here instead of re-explaining the plant. Sections 3.1 and 3.2 define the mechanism, its coordinate spaces, and the mappings between them. Section 3.3 fixes which signals exist, which are commanded, and which are merely estimated. Section 3.4 summarizes the high-fidelity Simscape model that generates all training data, and Section 3.5 the IsaacLab environment in which policies are trained and evaluated. Section 3.6 assembles the hierarchical control architecture, and Sections 3.7 and 3.8 state the control problem and the criteria against which the thesis will judge itself.

3.1 Manipulator Platform

The MULAG FME 500 is modeled as a serial-link manipulator with five revolute joints. Figure 3.1 shows the kinematic topology. The chain starts at the base frame bolted to the Unimog's front attachment plate (link 0), continues through the slew column (link 1), the two boom segments (links 2 and 3), and the head carrier (link 4), and ends in the flail mowing head (link 5). Four double-acting hydraulic cylinders actuate the first four joints; each cylinder closes a triangle over its joint, so cylinder extension and joint angle are related through the linkage geometry. The fifth joint, the rotation of the mowing head itself, carries no cylinder of its own in the configurations studied here: it is constrained by a swivel-link mechanism coupled to the head carrier, and the control task treats it as passive. Throughout the thesis the four actuated cylinders are indexed in the fixed order introduced in Chapter 5: slew (*Drehzapfen*), boom I (*Ausleger I*), boom II (*Ausleger II*), and head pivot (*Messerkopf-Schwenk*).

The geometry is not hand-approximated. Link shapes, masses, centers of mass, and full inertia tensors were extracted from the manufacturer's CAD data using the Blender

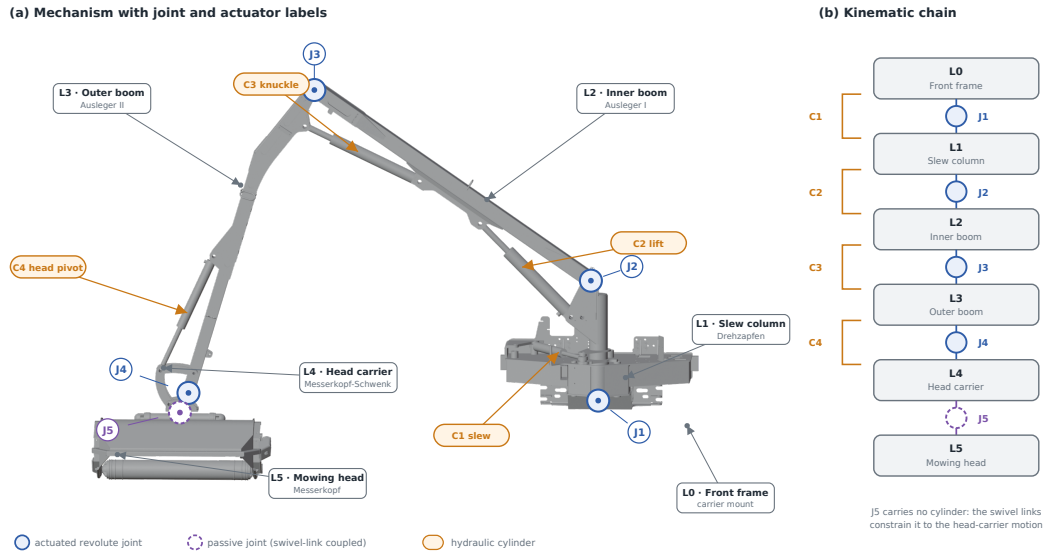


Figure 3.1: Kinematic topology of the MULAG FME 500, rendered from the CAD model. (a) The mechanism with every link (L0–L5), joint (J1–J5), and cylinder (C1–C4) called out; J5 is drawn dashed to mark it as passive. (b) The same structure as an abstract chain: five revolute joints in series, four of them each bracketed by the cylinder that actuates it, and the passive fifth joint coupled through swivel links rather than a cylinder of its own.

add-on Phobos [von Szadkowski 20], which exports a consistent URDF description of the mechanism; Figure 3.2 renders this CAD-derived model directly. This matters for a machine like the FME 500: the boom is a long, asymmetric weldment whose cross-inertia terms are significant, and a lumped-mass approximation would misrepresent the very load coupling that makes the hydraulics hard to control. The same CAD-derived description is the common ancestor of both simulated representations used in this thesis, the Simscape model of Section 3.4 and the IsaacLab articulation of Section 3.5, so their mechanical parameters agree by construction.

Table 3.1 lists the actuated joints and the extension ranges of their cylinders as observed across the training data. Two geometric facts recur throughout the thesis. First, the controlled point is not the origin of the head body: the cutter contact point sits 0.79 m below the *Messerkopf* body origin, and all task-space errors are measured at this offset tool tip. Second, the mowing pose of interest keeps the head facing straight down; the head-pivot cylinder holds a dedicated reference extension (0.589 m) to maintain this orientation, which effectively reserves the fourth cylinder for the orientation task and leaves cylinders 1–3 for positioning.

To make the serial chain concrete, Figure 3.3 shows the individual rigid bodies as they are imported from the URDF, each rendered on its own in an orthographic view. Reading outward from the vehicle: the *front frame* (link 0) bolts to the Unimog attachment plate and carries the slew bearing; the *slew column* (*Drehzapfen*, link 1) rotates the whole arm about the vertical axis; the *inner* and *outer boom* segments (*Ausleger I* and *II*, links 2 and 3) are the long load-bearing weldments whose asymmetric cross-sections give the significant cross-inertia terms noted above; the *head carrier* (*Messerkopf-Schwenk*, link 4)

Table 3.1: Actuated joints of the modeled FME 500 and the extension ranges of their cylinders observed in the training data (Chapter 4).

i	Joint (German designation)	Function	Extension range [m]
1	*Drehzapfen*	slew about vertical axis	0.018 – 0.493
2	*Ausleger I*	inner boom elevation	0.275 – 0.791
3	*Ausleger II*	outer boom articulation	0.309 – 0.857
4	*Messerkopf-Schwenk*	head pivot (orientation)	0.269 – 0.689
5	*Messerkopf*	head rotation, passive (swivel links)	—

with its swivel-link pair sets the head orientation; and the *flail mowing head* (link 5) does the cutting. Figure 3.3(g) shows one of the four double-acting cylinders as a separate barrel–rod pair, the form in which each actuator enters the closed kinematic loop of Section 3.2.

3.2 Kinematic Structure and Coordinate Mappings

3.2.1 Joint space and cylinder space

The mechanism's configuration is described by the joint vector $\boldsymbol{q} \in \mathbb{R}^5$ and its velocity $\dot{\boldsymbol{q}}$. The actuated coordinates, however, are the cylinder extensions $\boldsymbol{x} \in \mathbb{R}^4$ introduced in Chapter 5. Each cylinder connects an attachment point $\boldsymbol{p}_A$ on link k to a point $\boldsymbol{p}_B$ on link $k{+}1$, both fixed in their links and exported from the CAD data. The instantaneous cylinder length and extension are

$$\ell_i(\boldsymbol{q}) = \Big\| \boldsymbol{p}_B(\boldsymbol{q}) - \boldsymbol{p}_A(\boldsymbol{q}) \Big\|, \qquad x_i = \ell_i(\boldsymbol{q}) - \ell_{0,i}, \tag{3.1}$$

with $\ell_{0,i}$ the retracted reference length. Differentiating (3.1) gives the scalar mapping between joint motion and extension rate,

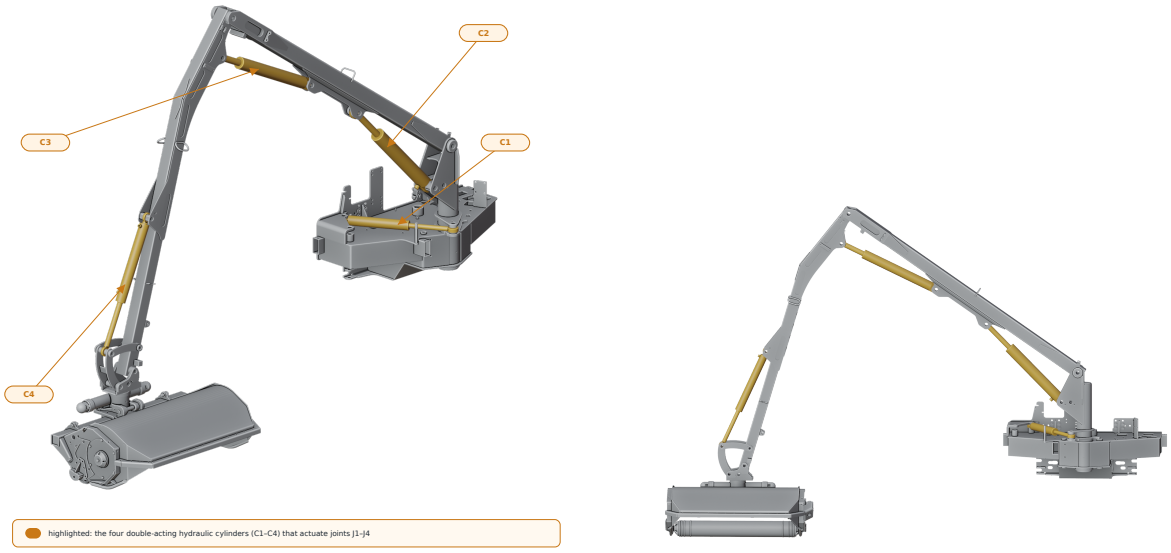
$$\dot{x}_i = J_{\mathrm{cyl},i}(\boldsymbol{q})\, \dot{\boldsymbol{q}}, \qquad J_{\mathrm{cyl},i} = \boldsymbol{e}_i^{\top} \Big(\boldsymbol{J}_B - \boldsymbol{J}_A \Big), \tag{3.2}$$

where $\boldsymbol{e}_i$ is the unit vector along the cylinder axis and $\boldsymbol{J}_A, \boldsymbol{J}_B$ are the translational Jacobians of the attachment points. The same mapping carries force back to torque: by the principle of virtual work, a cylinder force $F_{h,i}$ contributes the joint torque $\boldsymbol{\tau}_{\mathrm{act}} = \boldsymbol{J}_{\mathrm{cyl}}^{\top} \boldsymbol{F}_h$. The rows of $\boldsymbol{J}_{\mathrm{cyl}}$ are the lever arms discussed qualitatively in Chapter 2; they vary over the stroke, and with them the effective gain and stiffness that each cylinder presents to its joint.

3.2.2 Task space and the calibrated cutter Jacobian

The task is defined at the cutter tip, expressed in the truck chassis frame (`base_link`). This choice is deliberate and matters in practice: the articulation root of the simulated arm is the slew base, which moves with the arm, and using it as the reference frame corrupts every tracking measurement. All references and errors in this thesis live in the truck frame, and the controlled point includes the $-0.79\,\mathrm{m}$ tool-tip offset of Section 3.1.

For control, the composition of task kinematics and cylinder linkage is captured in a single calibrated Jacobian $\boldsymbol{J}_c \in \mathbb{R}^{3\times 4}$ that maps cylinder extension rates to cutter-tip



(a) (b)

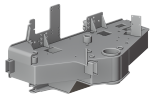
Figure 3.2: The CAD-derived multibody model of the FME 500 arm, rendered from the URDF export used throughout this thesis: (a) isometric view with the four hydraulic cylinders C1–C4 highlighted in orange and labeled, (b) side elevation in the folded zero pose. The swivel links of the passive fifth joint connect the head carrier to the flail head, and the base frame carries the slew column. The same geometry, masses, and inertia tensors drive both the Simscape multibody model and the IsaacLab articulation.

velocity in the truck frame. Rather than deriving it symbolically through the CAD chain, it was measured directly on the simulated plant: starting from the head-down mowing configuration, each cylinder target was stepped in turn and the settled cutter displacement recorded, giving central-difference estimates of every column. The measured matrix has rank 3 with singular values 8.24, 5.69, and 2.79, confirming that at the mowing operating point the four cylinders span exactly three independent task directions. This is the quantitative form of the observation from Section 3.1: with cylinder 4 committed to holding the head down, the remaining freedom moves the cutter in a three-dimensional workspace, and one direction of the nominal base frame is reachable only through vehicle motion. The damped least-squares controller of Chapter 6 inverts exactly this measured $\boldsymbol{J}_c$, so the nominal layer and the plant share one geometric contract.

3.3 Actuation and Measured State

Every interface in this thesis draws from one signal inventory, and it is worth stating it once, together with who can see what. Table 3.2 summarizes the quantities, their dimensions, and their availability in the two simulation environments.

The separation encoded in that table drives several design decisions. The valve command $\boldsymbol{u}$ exists only inside the Simscape plant and its data exports; in IsaacLab no valve is simulated, so any model that needs valve information at runtime must either receive it from a learned inverse model (the PPO1 path) or predict it internally (the PPO2 reduced-order model). The pressure difference $\Delta\boldsymbol{p}$ likewise has no native counterpart in the rigid-body



(a) Front frame (link 0)



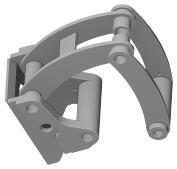
(b) Slew column (link 1)



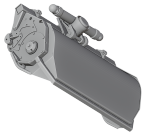
(c) Inner boom (link 2)



(d) Outer boom (link 3)



(e) Head carrier (link 4)



(f) Mowing head (link 5)



(g) Cylinder (barrel + rod)

Figure 3.3: Catalog of the FME 500 rigid bodies, each rendered individually from the URDF geometry in an orthographic view. Links 0–5 form the serial chain; (g) shows one of the four hydraulic cylinders as the barrel–rod pair that actuates a joint. Renders share the coordinate frames, scale, and matte engineering style used throughout the thesis.

simulation; at runtime it is a recursive state of the learned actuator models rather than a measurement. Only positions and velocities, of cylinders, joints, and the cutter tip, are common ground between the two worlds, which is why the acceptance gates of Section 3.8 are formulated on exactly these quantities. Finally, commanded quantities carry a star and estimated ones a hat, following the notation of Chapter 5; the latent $\hat{\boldsymbol{z}}$ exists only in the adaptive PPO2-C configuration.

3.4 High-Fidelity Simscape Model

The data source for every learned model in this thesis is a multi-domain Simscape model of the FME 500 that I developed in the master project preceding this thesis. This section summarizes the parts of that model the thesis depends on; the modeling and validation work itself is part of the contribution of this research program.

3.4.1 Mechanical subsystem

The mechanism is simulated as a rigid multibody system in Simscape Multibody [The MathWorks, Inc. 24 with the CAD-derived links, joints, and cylinder attachment frames of Section 3.1. The joint-space dynamics take the standard manipulator form

$$\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q},\dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) + \boldsymbol{\tau}_{\mathrm{fric}}(\dot{\boldsymbol{q}}) = \boldsymbol{\tau}_{\mathrm{act}} - \boldsymbol{J}^{\top}(\boldsymbol{q})\,\boldsymbol{F}_{\mathrm{ext}}, \tag{3.3}$$

Table 3.2: Signal inventory of the modeled system. "S" marks signals present in the Simscape data exports, "I" signals available live in the IsaacLab environment, and "M" quantities that exist there only through the learned models of Chapter 5.

Signal	Dimension	Availability	Meaning
$\boldsymbol{u}$	$[-1,1]^4$	S	normalized proportional-valve commands
$\boldsymbol{x}$, $\dot{\boldsymbol{x}}$	$\mathbb{R}^4$	S, I	cylinder extensions and velocities
$\Delta\boldsymbol{p}$	$\mathbb{R}^4$	S, M	chamber pressure differences
$\boldsymbol{F}$	$\mathbb{R}^4$	S	logged rod forces
$\boldsymbol{q}$, $\dot{\boldsymbol{q}}$	$\mathbb{R}^5$	S, I	joint positions and velocities
$\boldsymbol{p}_c$	$\mathbb{R}^3$	S, I	cutter-tip position, truck frame
$\boldsymbol{c} = [\Delta\boldsymbol{x}^*, \Delta\dot{\boldsymbol{x}}^*]$	$\mathbb{R}^8$	I	desired cylinder command
$\hat{\boldsymbol{z}}$	$\mathbb{R}^{d_z}$	M	estimated latent plant descriptor (PPO2-C)

with the inertia matrix $\boldsymbol{M}$ assembled from the imported CAD tensors. Two terms deserve comment. The friction torque $\boldsymbol{\tau}_{\text{fric}}$ combines viscous and Coulomb components with a smooth tanh approximation of the sign function, extended by a Stribeck term at the cylinder level to capture stick–slip at low velocities; the slew joint, which carries high side loads, is particularly sensitive to this. The external wrench $\boldsymbol{F}_{\text{ext}}$ models the vegetation cutting load as a nominal force plus a disturbance that scales with feed velocity and vegetation density, rather than as a constant, so mowing does not reduce to a static payload. Each cylinder is built as a closed kinematic loop (spherical–prismatic–cylindrical, shown in the top band of Figure 3.5), which lets it self-align under load without over-constraining the mechanism, and hard-stop constraints enforce the physical stroke limits of Table 3.1.

3.4.2 Hydraulic subsystem

Figure 3.4 shows the assembled plant at system level. The hydraulic side is deliberately physical rather than phenomenological. Supply comes from a fixed-displacement pump driven at variable speed ($Q_p = D_p\,\omega$), buffered by a gas-charged accumulator (polytropic gas model) and protected by a pressure-relief valve, so supply pressure is a dynamic quantity that sags under simultaneous demand and clips under load spikes. Each of the four actuators is a 4/3 proportional directional valve driving an asymmetric double-acting cylinder; Figure 3.5 details one such channel, from the rigid-body assembly of barrel and rod through the mechanical–hydraulic coupling interface down to the valve, the chambers, and the measured feedback signals. The chamber pressures follow the continuity equations

$$\dot{p}_1 = \frac{E_{\text{eff}}}{V_1(x)}\Big(Q_1 - A_1\dot{x} - C_{\text{leak}}(p_1 - p_2)\Big), \qquad \dot{p}_2 = \frac{E_{\text{eff}}}{V_2(x)}\Big(-Q_2 + A_2\dot{x} + C_{\text{leak}}(p_1 - p_2)\Big), \tag{3.4}$$

with stroke-dependent chamber volumes $V_{1,2}(x)$ that include the line dead volumes, piston areas $A_1 \neq A_2$, and laminar internal leakage C_{leak}. The effective bulk modulus accounts for entrained air through

$$E_{\text{eff}}(p) = \left(\frac{1}{E_f} + \frac{\alpha}{p}\right)^{-1}, \tag{3.5}$$

so fluid stiffness itself is pressure dependent, weakest exactly where precise low-pressure positioning is attempted. Valve flow follows the orifice law of Chapter 2 (Equation 2.1), and

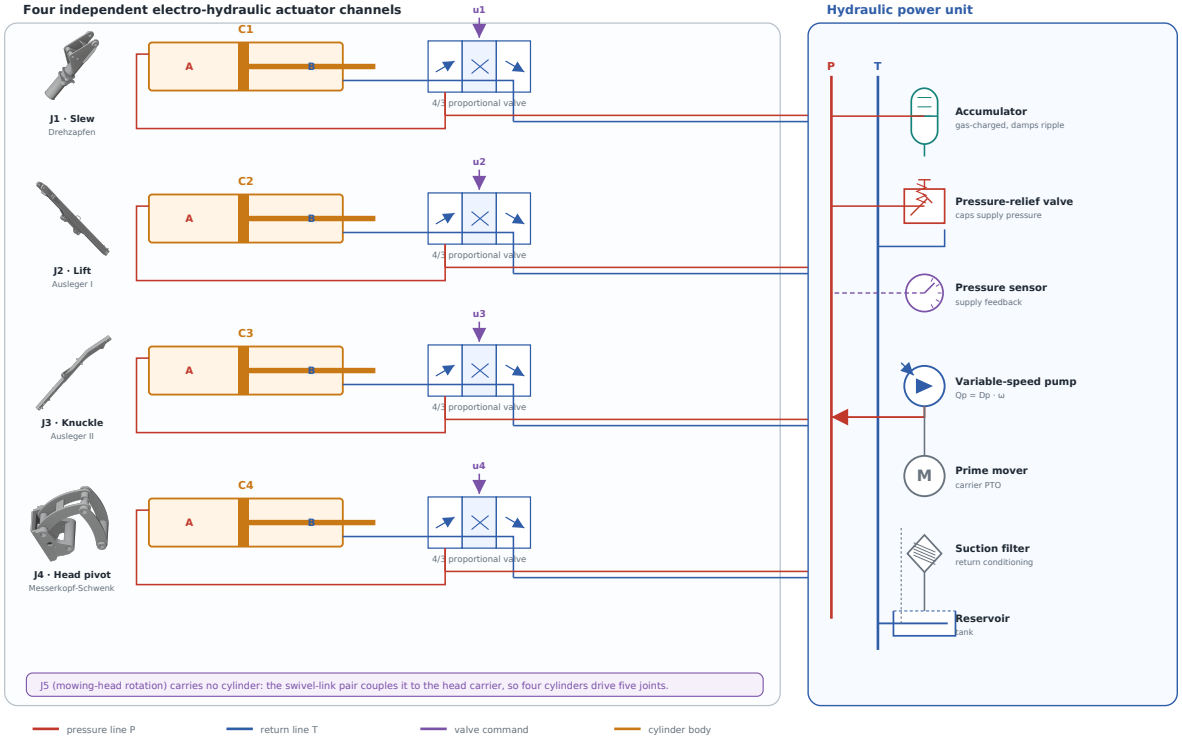


Figure 3.4: System-level hydraulic schematic of the modeled machine. Left: the four independent electro-hydraulic actuator channels, each a 4/3 proportional directional valve driving one double-acting cylinder that moves its link (thumbnails rendered from the same CAD model as Figures 3.1–3.3). Right: the shared hydraulic power unit, comprising the variable-speed pump and its prime mover, the gas-charged accumulator, the pressure-relief valve, the pressure sensor, the suction filter, and the reservoir, connected to every channel through the common supply (P) and return (T) rails. Only four cylinders are present: the fifth joint is passive (Table 3.1) and carries no actuator of its own.

the spool is not an instantaneous gain but a first-order lag, $\dot{x}_v = (u - x_v)/\tau_v$, so command transients are bandwidth limited. The net piston force $F_h = p_1 A_1 - p_2 A_2 - F_{\text{fric}}(\dot{x})$ feeds back into the mechanism through the virtual-work mapping of Section 3.2, closing the bidirectional coupling: mechanical loads shape pressure transients, and pressure transients shape motion. Because cylinder-induced stiffness is high while rigid-body motion is slow, the coupled equations are stiff, and the model runs under a variable-step implicit solver; this stiffness is the computational reason the Simscape plant cannot serve as an RL training environment (Section 2.1.3).

3.4.3 What the model is validated to reproduce

A plant model intended as a data generator must be trusted for its *dynamics*, not just its geometry. The validation strategy in the modeling study was control-based: a fixed integral backstepping controller at 0.01 s was applied as a standardized excitation instrument, and the emergent behavior of the closed loop was examined as the physical parameters varied. The headline experiment sweeps the effective bulk modulus over $\beta \in \{0.5, 1.0, 1.5, 2.0\}$ GPa, the physically meaningful range from strongly aerated to very stiff fluid, with all controller

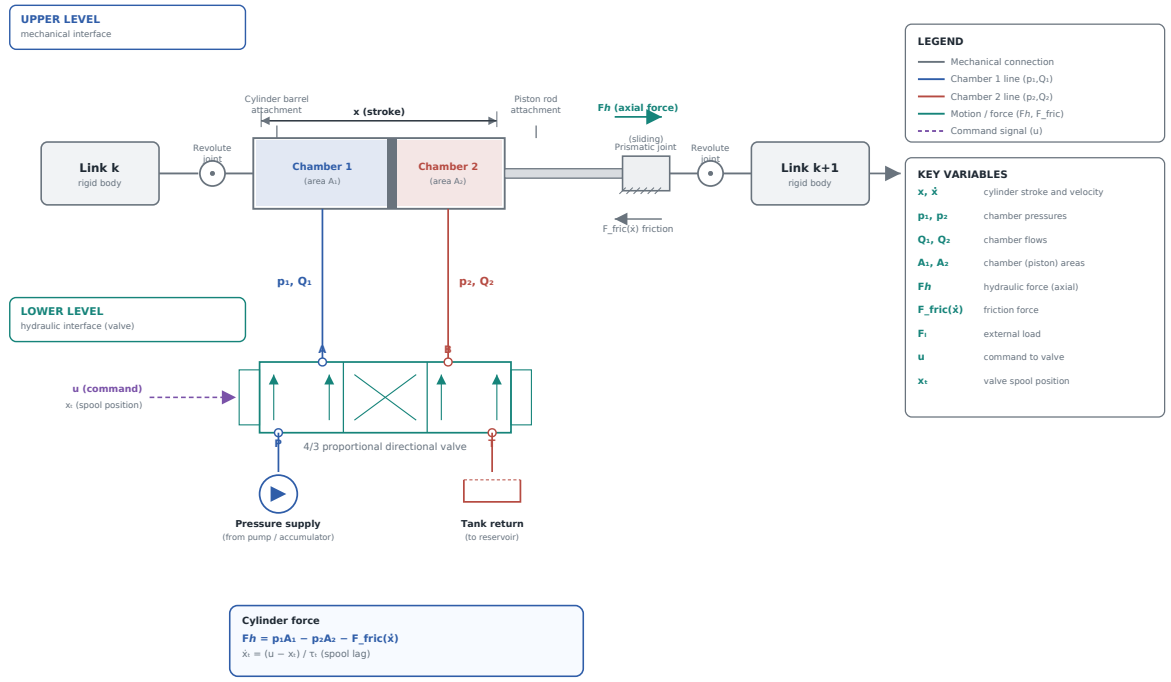


Figure 3.5: One electro-hydraulic actuator channel, split into the upper mechanical interface and the lower hydraulic interface that Sections 3.2 and 3.4 describe. Top: barrel and rod are rigid bodies connected to the adjacent links through a revolute–prismatic–revolute joint chain whose prismatic coordinate is the stroke x; the prismatic joint is the translational interface that exchanges the axial force F_h against the motion $x, \dot{x}$, opposed by the friction $F_{\text{fric}}(\dot{x})$. Bottom: the asymmetric double-acting cylinder, with chamber pressures and flows (p_1, Q_1) and (p_2, Q_2) routed to ports A and B of the 4/3 proportional directional valve, which is driven by the command u through the first-order spool lag and connected to the shared supply and return rails of Figure 3.4. The legend, variable table, and force identity use the same notation as the continuity equation (3.4) and the F_h identity of Section 3.4.

gains held constant. Figure 3.6 shows the result re-plotted from the study's measurements. Actuation delay behaves as hydraulic intuition demands: the valve-to-joint delay of the heavy slew axis falls from about 0.85 s at 0.5 GPa to 0.65 s at 2.0 GPa, the system-level delay index from roughly 0.45 s to 0.35 s, and the improvement saturates beyond 1.5 GPa, where mechanical inertia rather than fluid compliance limits the response. Task-space tracking error falls monotonically with stiffer fluid, by about 11 % over the sweep. The model, in other words, reproduces the "stiff yet oscillatory" character of real hydraulic machinery and responds to parameter changes in the physically expected direction, which is precisely the property that Chapter 4 exploits when it randomizes these same parameters to generate plant families.

Three limitations are carried forward honestly. The model is a single machine: manufacturing spread enters only through the deliberate parameter randomization of Chapter 4, not through measured variation. Thermal dynamics are not modeled; temperature effects are represented indirectly through the parameters they act on (bulk modulus, friction). And the logged rod-force channel failed a piston-area consistency audit on the current export (Section 5.4), so force is treated as unreliable supervision throughout the thesis.

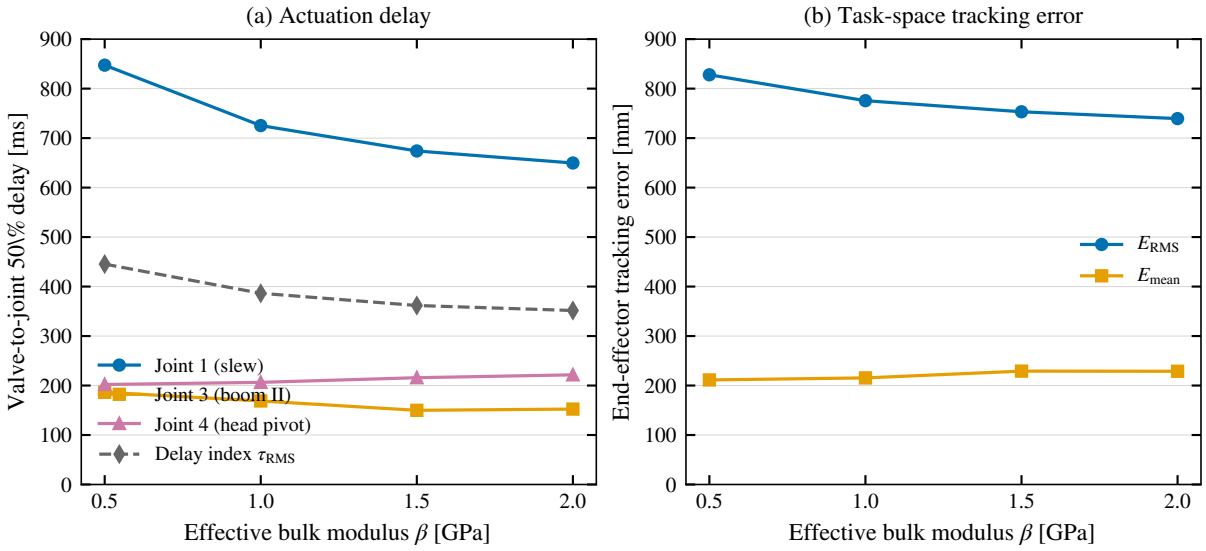


Figure 3.6: Sensitivity of the Simscape plant to the effective bulk modulus, measured in the preceding modeling study. (a) Valve-to-joint 50 % delays of the dominant axes and the RMS delay index; (b) end-effector tracking error of the fixed validation controller. Softer fluid slows the arm and degrades tracking; the effect saturates beyond 1.5 GPa.

3.5 IsaacLab Environment

Policy training and closed-loop evaluation run in IsaacLab [Mittal 23] on the GPU-parallel PhysX engine. The same CAD-derived mechanism enters here as a USD articulation: the Unimog carrier with wheels, the five arm links, and the four cylinders represented as prismatic actuator joints, with self-collisions enabled and the solver configured for the long-chain mechanism (eight position iterations). The physics step is 1/120 s with a control decimation of two, so every controller in this thesis acts at 60 Hz; the learned actuator models keep their native 0.05 s sample time from the Simscape data, and their outputs are resampled onto the control clock. Training uses 64 parallel environments, the measured throughput optimum on the available GPU.

Three environment details cost real debugging time and are recorded here so they need not be rediscovered. First, the articulation root of the arm sits about 1.11 m above the wheel contact plane, so the vehicle must spawn with its root at $z = 1.26$ m; spawning at ground height buries the truck to its axles and silently changes every contact force. Second, cylinder actuation uses PhysX position targets, and the target must be integrated in *reference* space, accumulating commanded increments, rather than re-anchored each step to the measured position: re-anchoring lets gravity sag wash out every commanded increment, removing the policy's authority entirely. Third, during arm tracking the vehicle is parked by holding the wheel positions with a planar fixture (with a small deadband), because wheel compliance otherwise turns arm reaction forces into slow truck drift that contaminates the task-space error. The full hydraulic model does not run inside these environments for the computational reasons of Section 2.1.3; its behavior enters only through the learned actuator representations between the controller and the PhysX targets.

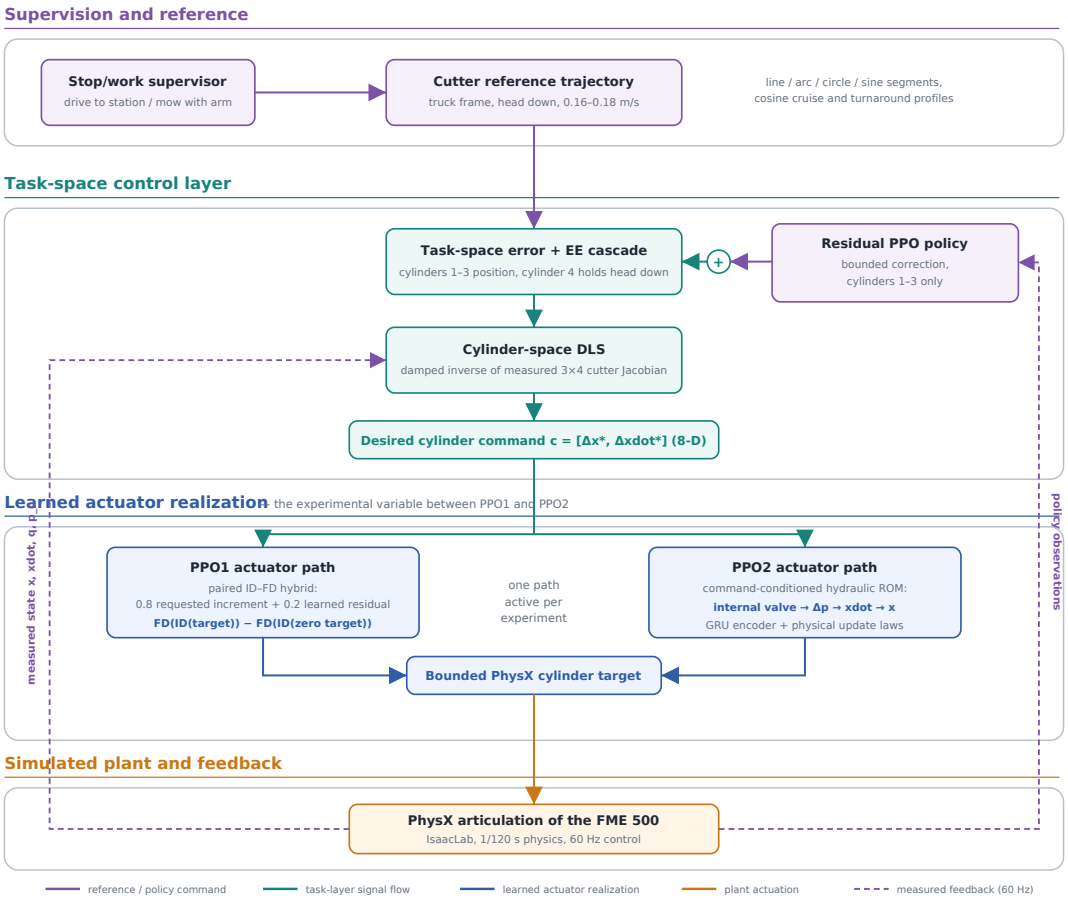


Figure 3.7: Hierarchical control architecture used in all experiments, organized by layer. A stop/work supervisor alternates vehicle driving and arm tracking, and the reference generator supplies the head-down cutter trajectory in the truck frame. The task layer converts the reference into the desired cylinder command through the EE cascade and the damped least-squares inverse of the measured cutter Jacobian, with the residual PPO policy adding a bounded correction for cylinders 1–3. The learned actuator layer is the experimental variable: either the paired ID–FD hybrid (PPO1) or the command-conditioned hydraulic ROM (PPO2) realizes the command as a bounded PhysX position target. The measured state closes both loops at 60 Hz.

3.6 Hierarchical Control Architecture

Figure 3.7 assembles the pieces. At the top, a stop/work supervisor alternates between driving the vehicle to a station (arm stowed) and tracking with the arm (vehicle parked); all tracking results in this thesis are scored only during the arm-tracking phase. The task layer turns the cutter reference into cylinder-space commands: the head-down orientation is held by the dedicated cylinder-4 reference, an end-effector cascade distributes the position task over cylinders 1–3, and the damped least-squares inverse of the measured Jacobian $\boldsymbol{J}_c$ produces the desired increments with a velocity-scaled lead and bounded per-step magnitude. The residual PPO policy observes the tracking state and adds a bounded correction for cylinders 1–3; it cannot command cylinder 4 or the vehicle. Below the task layer sits the experimental variable of the whole thesis: the actuator path that realizes the desired command, either the paired ID–FD hybrid (PPO1) or the command-conditioned

reduced-order model (PPO2), both ending in the same bounded PhysX position target. The measured state closes both loops, feeding the DLS layer and the policy at 60 Hz.

3.7 Problem Formulation

With the system fixed, the control problem can be stated compactly. Let $\boldsymbol{p}_{\mathrm{ref}}(t)$ be a reference trajectory for the cutter tip in the truck frame, drawn from the trajectory classes used throughout the thesis (lines, arcs, circles, and sine sweeps at cruise speeds of 0.16–0.18 m/s, with cosine acceleration and turnaround profiles), and let $\boldsymbol{p}_c(t)$ be the measured tip position. The instantaneous task error is

$$\boldsymbol{e}(t) = \boldsymbol{p}_{\mathrm{ref}}(t) - \boldsymbol{p}_c(t). \tag{3.6}$$

The *tracking objective* is to keep $\|\boldsymbol{e}(t)\|$ small in mean and in the tail (p95) over complete reference cycles, while the head stays within its orientation band and the motion remains smooth. The *model objective*, developed in Chapter 5, is that each learned actuator representation predict the cylinder response to its commands accurately over one step and over recursive rollouts, with demonstrated input authority. The *robustness objective* asks that tracking persist across the randomized plant family $\{\mathcal{P}(\boldsymbol{\xi})\}$ of Chapter 4, and the *adaptation objective* that the latent-conditioned configuration recover performance after the active $\boldsymbol{\xi}$ changes, faster and further than the robust configuration alone. All objectives are subject to the same constraint set: valve commands within the per-axis limits of Chapter 5, cylinder extensions within the ranges of Table 3.1, per-step target increments bounded (the 25 mm cylinder-target lead limit), head-down alignment never below 0.95, and the parked vehicle essentially motionless.

What makes this problem hard is not any single ingredient but their conjunction, and the chapters so far have each contributed one piece of the difficulty: the actuator chain is nonlinear and drifting (Chapter 2), only three task directions are controllable and the mapping into them varies with configuration (Section 3.2), the true actuator state is not observable at runtime (Section 3.3), and the high-fidelity plant that knows the answers is too slow to train against (Section 3.4).

3.8 Evaluation Criteria at System Level

The thesis judges controllers and models by fixed, pre-registered gates rather than by single aggregate scores, and it separates the two levels deliberately: a controller metric can hide model weakness behind policy compensation (Section 2.7), so model metrics are always reported alongside.

At the *system level*, an accepted tracking run must complete its reference cycles and satisfy, at minimum: mean and p95 tool error improved over the zero-residual baseline, bounded maximum error, at least 99 % occupancy of the ground clearance band, head-down alignment of at least 0.995 during scored motion, truck planar drift below 2 mm, and a filtered path-length ratio between 0.95 and 1.25. Two smoothness criteria guard against a failure mode that plain error metrics miss, a policy that tracks on average while oscillating: the p95 of the detrended tool oscillation must stay below 10 mm and the high-frequency excess path below 15 %. At the *cylinder level*, step gates require clean response to fresh

$\pm 10\,\mathrm{mm}$ commands on every axis, small terminal error on large ($80\,\mathrm{mm}$) coordinated motions, and low out-and-back return error, all without valve-limit saturation. Model-level criteria, one-step and rollout accuracy, input-ablation authority, and Jacobian conditioning, were introduced with the models in Chapter 5. Chapter 7 instantiates all three families of criteria in the full experimental matrix.

The system is now fully specified: the machine, its two simulated representations, the signals that connect them, and the objectives and gates that define success. The next chapter turns to the data that links the two representations, the Simscape campaigns from which every learned model in this thesis is trained.

4. Data Generation and Preparation

Chapter 3 fixed the plant: the mechanism, the two simulated representations of it, and the signals that connect them. What it did not fix is how those two representations are supposed to meet. The Simscape model knows the hydraulics but is far too slow to train a policy against; the IsaacLab articulation is fast but has no hydraulics at all. Everything the learned models of Chapter 5 are able to say about the actuator has to travel from the first world to the second inside a dataset. This chapter is about how that dataset was designed, collected, audited, and prepared.

Two collection campaigns are described. The first drives the arm with a closed-loop controller through a large randomized curriculum and produces the realistic operating distribution that the inverse model and the reduced-order model learn from. The second removes the controller entirely and applies deliberate excitation to the valves, because the closed-loop campaign turned out to be unable, in principle, to identify what the valve does. That second finding is the reason this chapter is longer than a dataset description usually needs to be. It is also, in my view, one of the more transferable results of this thesis, so Section 4.4 sets out the diagnosis and the repair in full rather than presenting only the final data.

4.1 Dataset Objectives and Design Requirements

Before any run was collected I fixed four requirements that the data had to satisfy jointly. They are worth stating explicitly, because the rest of the chapter is essentially the record of discovering that no single collection strategy meets all four.

R1: realistic operating distribution. The learned models are deployed inside a tracking controller, so they must be accurate in the states that controller actually visits: coordinated multi-axis motion, gravity-loaded poses, moderate speeds. Data collected far from that distribution would be accurate where it does not matter.

R2: causal identifiability of the valve. A forward model is only useful to a policy if it responds to the command. This requires that the valve signal in the training data

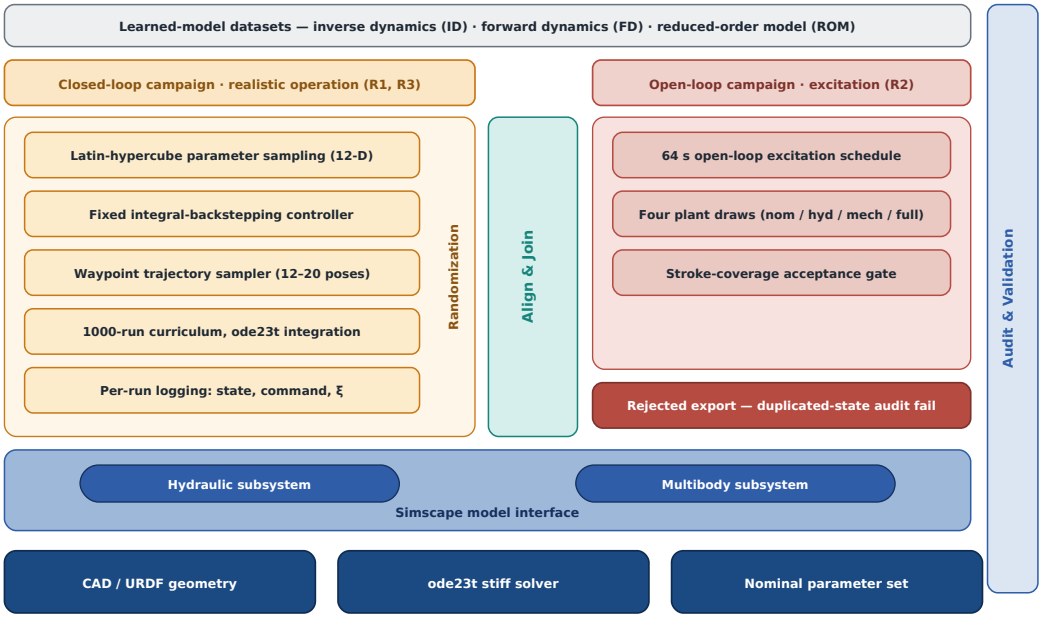


Figure 4.1: Architecture of the data-generation system. Both campaigns sit on the shared Simscape substrate (bottom) and feed the learned-model datasets (top). The closed-loop stack (left) produces realistic randomized operation for requirements R1 and R3; the open-loop stack (right) produces causal excitation for requirement R2, with a coverage gate and an audit that rejected an earlier export. The two are reconciled by the alignment and join stage, and every stage is checked by the audit spanning the right edge. The rest of this chapter walks up this diagram layer by layer.

carry information that the state history does not already carry, which is a property of the *experiment*, not of the network [Ljung 99, Isermann 11].

R3: variation across a plant family. Robust training and, later, latent adaptation both need many plants rather than one. The variation has to be physically meaningful and it has to be recorded per run, because the teacher latent of Section 5.5 is computed from it.

R4: clean separation. Train, validation, and test material must be separable at the level of complete runs, so that no transition window straddles a split boundary and no run contributes to two splits.

R1 and R2 pull against each other, and that tension is the organising idea of this chapter. A controller that produces realistic motion computes its command from the measured state, which makes command and state history redundant and destroys identifiability (Section 2.4.2). An experiment that guarantees identifiability has to break the feedback path, which immediately makes the motion unrealistic. Rather than compromise on one of them, I collected both: a closed-loop campaign that satisfies R1 and R3, and an open-loop campaign that satisfies R2. Section 4.5 explains how the two are used together without letting either one dominate.

Table 4.1: Randomized hydraulic parameters. The seal-wear coordinate is drawn independently for each of the four cylinders and drives three physically linked quantities at once.

Parameter	Nominal	Randomized range	Acts on
Entrained air fraction ε_{air}	0.005	0.001 – 0.080	fluid compliance
Dead-volume fraction ε_{vol}	0.020	0.005 – 0.080	fluid compliance
Discharge coefficient c_d	0.70	0.56 – 0.84	valve flow gain
Seal wear $w_{\text{seal},i}$	0	0 – 0.25 (per cylinder)	clearance, leakage, spool lag
Relief setting p_{set}	27.6 MPa	24.8 MPa – 30.4 MPa	pressure ceiling

4.2 Randomized Parameter Space

4.2.1 Sampling design

The closed-loop campaign draws its plants from a twelve-dimensional parameter space using Latin hypercube sampling [McKay 79], which spreads the draws more evenly over each marginal than independent uniform sampling would at this sample size. The twelve coordinates are fixed once for the whole batch, before any simulation runs, so the parameter table is reproducible and can be audited independently of the data it produced.

The batch is organised as a curriculum rather than a single homogeneous pool. Run 1 is the untouched nominal plant. Runs 2–251 randomize only the hydraulics, runs 252–501 only the mechanics, and runs 502–1000 both together, giving the composition shown in Figure 4.2(a). This structure is deliberate: it makes it possible to attribute a later modelling failure to the hydraulic or the mechanical axis by retraining on one block at a time, which a uniformly mixed batch would not allow.

4.2.2 Hydraulic parameters

Table 4.1 lists the hydraulic axes. They were chosen to correspond to the physical mechanisms that make one machine differ from another in service, in the sense discussed in Section 2.1.2: air works its way into the fluid, spools and seals wear, and relief settings drift.

The seal-wear coordinate deserves comment because it is the one place where a single sampled number is deliberately allowed to move several model parameters together. A worn cylinder does not become leakier without also becoming slower, so $w_{\text{seal},i}$ scales the piston clearance linearly, the valve leakage area with the cube of the same factor, and the spool time constant from 8 ms to about 13 ms at the top of the range. Sampling those three independently would have produced physically incoherent machines, some of them very leaky and yet perfectly fast.

4.2.3 Compliance-equivalent representation of hydraulic compressibility

The two air-related coordinates need to be described carefully, because what they do in the model is not what their names suggest, and stating it loosely would misrepresent the data. The generation campaign uses an isothermal fluid with a fixed base bulk modulus

$$\beta_0 = 0.70\,\text{GPa}, \tag{4.1}$$

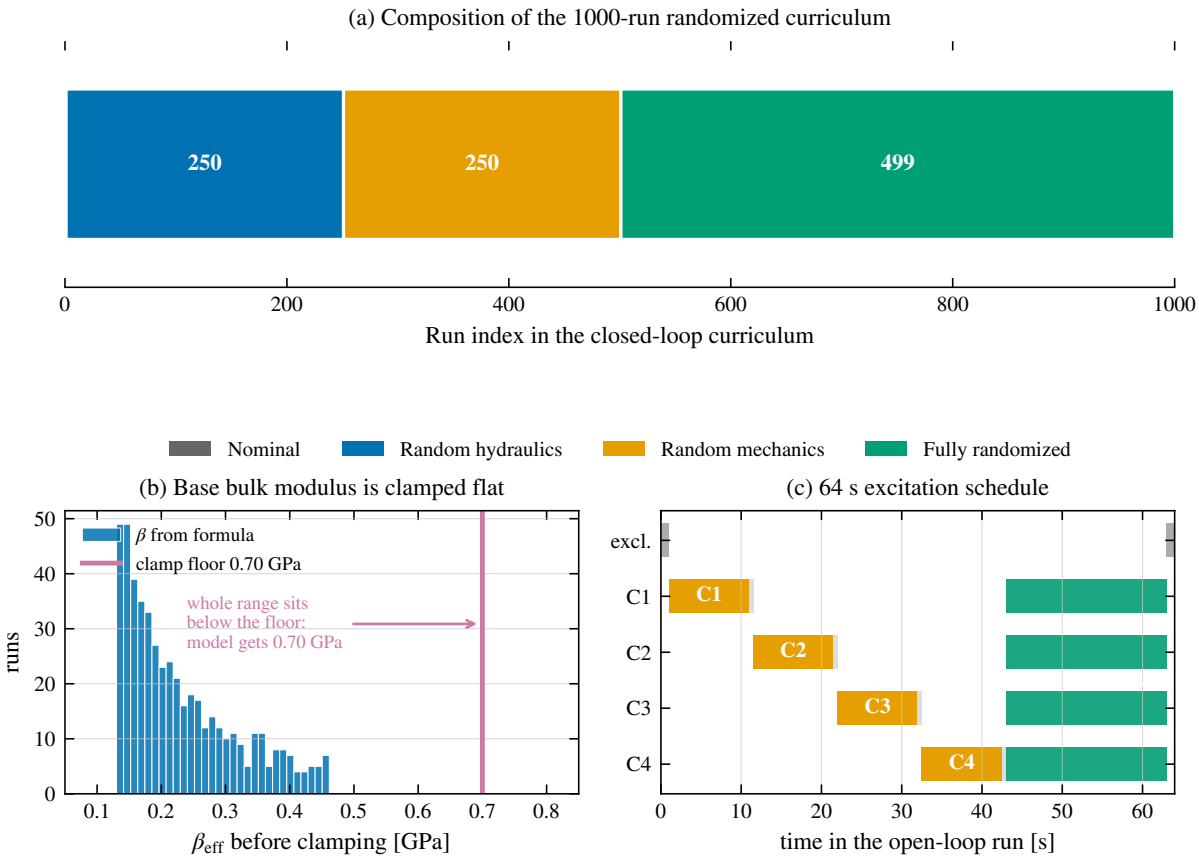


Figure 4.2: Design of the two collection campaigns. (a) Composition of the closed-loop curriculum: one nominal run, then 250 hydraulics-only, 250 mechanics-only, and 499 fully randomized draws. (b) Base bulk modulus from the entrained-air and dead-volume parameters. The series formula returns 0.13–0.46 GPa across the sampled range, entirely below the 0.70 GPa clamp floor, so the stored value is constant at 0.70 GPa on every run and the base fluid stiffness is not randomized; the same two coordinates still reach the plant weakly through the chamber dead volume (Section 4.2). (c) Schedule of the 64 s open-loop excitation run: a startup second that is excluded from training, four sequential single-cylinder slots separated by zero-command gaps, a simultaneous phase on all four channels, and a shutdown second.

held constant across all thousand runs, and auditing the stored run descriptors confirms that the applied value is exactly β_0 on every one of them. It would therefore be wrong to claim that the fluid bulk modulus was randomized. What the entrained-air and dead-volume coordinates actually randomize is the chamber *dead volume*,

$$V_{d,i} = V_{d,0}\left(1 + \varepsilon_{\mathrm{air},i} + \varepsilon_{\mathrm{vol},i}\right), \tag{4.2}$$

which is assigned to both chambers of run i; the stored values span 505 to 579 mL, an unclamped spread of about thirteen percent. The reason this matters is that the pressure dynamics do not depend on β_0 alone. With the stroke-dependent chamber volumes $V_{1,i}(x) = V_{d,i} + A_1 x$ and $V_{2,i}(x) = V_{d,i} + A_2(L - x)$, the continuity equation (3.4) carries the coefficient

$$\dot{p}_1 = \frac{\beta_0}{V_{1,i}(x)}\left(Q_1 - A_1\dot{x} - C_{\mathrm{leak}}(p_1 - p_2)\right), \qquad K_{p,i}(x) \equiv \frac{\beta_0}{V_i(x)}, \tag{4.3}$$

so it is the ratio β_0/V, not β_0, that sets the pressure-response rate. Randomizing $V_{d,i}$ therefore varies the same reduced-order coefficient that randomizing the modulus would have varied. The hydraulic stiffness reflected at the piston, $k_{h,i}(x) \propto \beta_0 A^2/V_i(x)$, and the associated natural frequency, $\omega_{h,i}(x) \propto \sqrt{\beta_0 A^2/(m_{\text{eq}} V_i(x))}$, move with it: a larger chamber is more compliant and slower, a smaller chamber stiffer and faster.

To give the realized variation a modulus-like label without the false claim, I reference each run to the nominal chamber and define a *compliance-equivalent bulk modulus*

$$\beta_{\text{eq},i} = \beta_0 \frac{V_{d,0}}{V_{d,i}}, \qquad \text{so that} \qquad \frac{\beta_{\text{eq},i}}{V_{d,0}} = \frac{\beta_0}{V_{d,i}}, \tag{4.4}$$

which preserves exactly the compressibility coefficient of (4.3). Equivalently, the run can be described by the pressure-stiffness index $K^d_{p,i} = \beta_0/V_{d,i}$ or the compliance index $C^d_{h,i} = V_{d,i}/\beta_0$. These are the correct learning and adaptation targets for the hydraulic compressibility axis, precisely because the stored β_{eff} is constant while β_{eq}, K^d_p, and C^d_h carry the variation that was physically present in the simulated trajectories. Their distribution is shown in Figure 4.3, alongside the other three hydraulic axes.

Two consequences follow, and both are stated as limitations rather than strengths. First, the equivalence holds only for the compressibility term. Changing the chamber volume reproduces the pressure-build-up coefficient that a modulus change would produce, but it does not reproduce temperature-dependent density, viscosity, or leakage; those were not modelled, and the dataset is isothermal. In a real machine, temperature, operating pressure, entrained air, hose expansion, and structural compliance all feed the effective compliance, and none of them is separately identifiable here. They are represented only through the single lumped coordinate $C_h = V/\beta$, or equivalently $K_p = \beta/V$, which is what the latent encoder of Section 5.5 is able to recover. Second, the choice of a fixed modulus was partly a computational one. Pushing β to large values makes the fluid nearly incompressible and lifts the hydraulic natural frequency, which sharpens the time-scale separation between the fast pressure modes and the slow rigid-body motion and turns the coupled differential-algebraic system stiffer; the variable-step solver then needs smaller steps and more frequent Jacobian updates. Multiplied across a thousand coupled hydraulic-multibody runs, that cost is the difference between an overnight campaign and a multi-day one. The fixed 0.70 GPa was chosen as the compromise that keeps genuine compressible pressure dynamics while holding the numerical stiffness, and the wall-clock cost, within budget.

The batch is therefore honestly described as containing randomized effective chamber compliance, or equivalently randomized hydraulic pressure stiffness, under an isothermal fluid assumption, and this thesis uses that language rather than the stronger and unsupported claim of a randomized fluid modulus.

4.2.4 Mechanical parameters

The mechanical axes, listed in Table 4.2, are applied to all fourteen rigid bodies of the mechanism: the six links, the four cylinder barrels, and the four pistons. For each body the mass is perturbed by up to ±20 %, the centre of mass by up to ±5 % of its nominal offset, and the inertia tensor together with its products of inertia is perturbed consistently

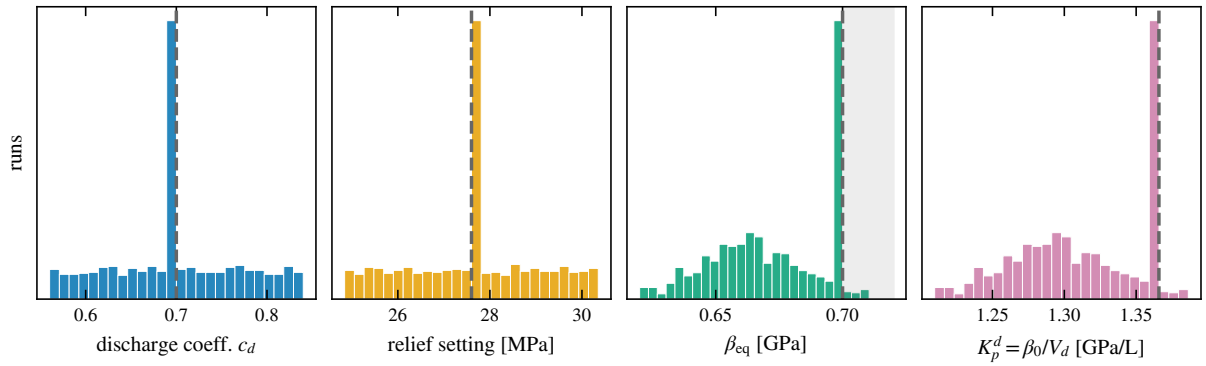


Figure 4.3: Hydraulic parameters as actually delivered to the plant, read from the stored descriptors of all 1 000 curriculum runs (dashed line: nominal value). The discharge coefficient and relief setting vary as designed. The rightmost panels show the compressibility axis in its correct form: the base modulus β_0 is constant, but the compliance-equivalent modulus $\beta_{\text{eq}} = \beta_0 V_{d,0}/V_d$ of (4.4) and the pressure-stiffness index $K_p^d = \beta_0/V_d$ carry the variation that the randomized dead volume actually produces in the pressure dynamics.

Table 4.2: Randomized mechanical parameters. The rigid-body perturbations are applied independently to each of the fourteen bodies.

Parameter	Nominal	Randomized range
Body mass	CAD value	±20 %
Centre-of-mass offset	CAD value	±5 %
Inertia tensor and products	CAD value	consistent with mass draw
Joint stiffness, J1 and J2	60 kN/m	40 – 120 kN/m
Joint stiffness, J3 and J4	60 kN/m	40 – 120 kN/m
Joint stiffness, J5	25 kN/m	15 – 60 kN/m
Joint damping	250	100 – 600

with the mass change, so that the body remains physically realisable rather than acquiring an inertia that its mass could not produce.

The joint stiffness and damping terms model compliance in the bearings and linkage rather than in the fluid. They matter here because they set how much of a pressure transient appears as boom oscillation instead of clean motion, which is exactly the coupling that a purely kinematic controller cannot anticipate.

4.3 Closed-Loop Data Collection

The complete closed-loop procedure is summarized in Algorithm 4.1. Two features of it are worth reading off before the prose expands on them: the parameter table is drawn in full before any simulation runs, so the plant family is fixed and reproducible independently of the collection, and the reference trajectory is resampled inside every run, so no two runs share a path.

Algorithm 4.1: Closed-loop curriculum generation.

input : run budget $N = 1000$; joint limits $\boldsymbol{q}_{\min}, \boldsymbol{q}_{\max}$; nominal parameters $\boldsymbol{\theta}_0$
output : dataset $\mathcal{D} = \{(\text{trajectory}_i, \boldsymbol{\xi}_i)\}_{i=1}^{N}$

$\mathbf{L} \leftarrow \textsc{LatinHypercube}(N, 12)$ `// one design for the whole batch`
for $i \leftarrow 1$ **to** N **do**
  assign i to a curriculum block (nominal / hydraulic / mechanical / full)
  $\boldsymbol{\xi}_i \leftarrow \textsc{DrawParameters}(\mathbf{L}_i, \text{block})$ `//` $\varepsilon_{\text{air}}, \varepsilon_{\text{vol}}, c_d, w_{\text{seal}}, p_{\text{set}}$ `; masses, CoM, inertia, stiffness`
end
for $i \leftarrow 1$ **to** N **do**
  inject $\boldsymbol{\xi}_i$ into the Simscape model
  $n_w \leftarrow \textsc{RandInt}(12, 20)$; $\boldsymbol{W} \leftarrow \textsc{LatinHypercube}(n_w, \text{JDOF})$ within $[\boldsymbol{q}_{\min}, \boldsymbol{q}_{\max}]$
  assign each segment a speed class (slow / medium / fast / variable); **no** dwell
  $\boldsymbol{q}_{\text{ref}}(t) \leftarrow \textsc{Interpolate}(\boldsymbol{W})$ over 22 s
  simulate with `ode23t`; log on the 0.05 s grid
  store $(\boldsymbol{u}, \boldsymbol{x}, \dot{\boldsymbol{x}}, \Delta\boldsymbol{p}, \boldsymbol{F}, \boldsymbol{q}, \dot{\boldsymbol{q}}, \boldsymbol{q}_{\text{ref}}, \boldsymbol{\xi}_i)$
  if $i \bmod 100 = 0$ **then** checkpoint $\mathcal{D}$ to disk
end

4.3.1 The controller as a data-collection instrument

Every closed-loop run is driven by the same fixed integral-backstepping controller, operating on all five joints at a 0.005 s rate. Its gains are never retuned between runs. This is deliberate: the controller is here as a standardised instrument for putting the arm through realistic motion, not as a control contribution of this thesis, and holding it fixed is what makes the resulting behavioural differences attributable to the plant draw rather than to the controller.

Three details of the controller matter later. The second joint carries the gravity load of the whole boom, so it is handled by a proportional-integral law with velocity damping and an explicit gravity bias rather than by the backstepping law used on the others. A barrier term keeps the commanded configuration away from the joint limits imported from the URDF. And the commands are saturated per axis at [0.95, 0.95, 0.80, 0.80, 0.90]. That last vector is worth noting: the first four entries are exactly the clip limits stored in the inverse-dynamics checkpoint in Section 5.2. The ID network inherited its output limits from the instrument that generated its training labels, which is correct but easy to mistake for a free design choice.

4.3.2 Trajectories

Each run receives its own reference trajectory, sampled rather than scripted. Between twelve and twenty waypoints are drawn by Latin hypercube sampling within the joint limits, and the arm moves between them without dwelling, so that the data contains transitions rather than long static holds. Segment durations are drawn from four speed classes, slow (1.5–2.5 s), medium (0.8–1.5 s), fast (0.3–0.7 s), and variable (0.5–2.5 s), and about a third of the runs receive additional exploratory perturbations on top of the waypoint path.

An earlier version of this generator used four to eight waypoints with dwell periods between them. It produced data in which the arm was stationary for a large fraction of the time,

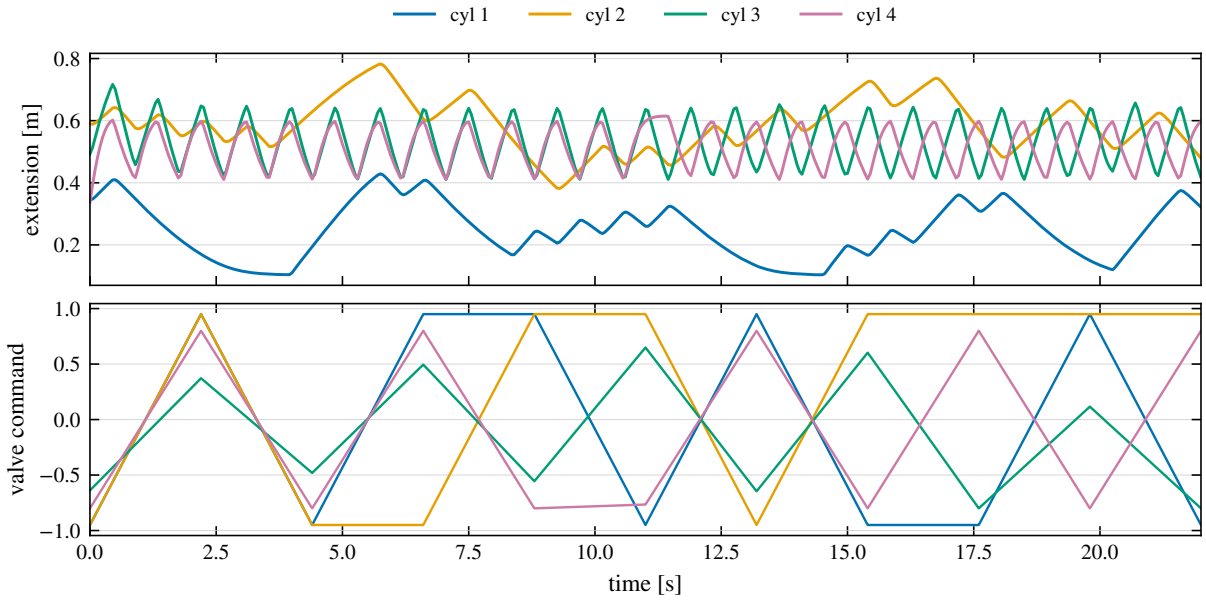


Figure 4.4: One closed-loop operating run (fully randomized draw, run 601 of the curriculum). Top: the four cylinder extensions, moving continuously and out of phase as the backstepping controller tracks the sampled waypoint trajectory. Bottom: the corresponding valve commands. The motion is coordinated and gravity loaded, which is the operating regime the learned models must be accurate in, but the valve command is nearly a function of the state that produced it, which is the property Section 4.4 shows makes the valve unidentifiable from this data alone.

which flatters any model that learns to predict persistence. The current settings were chosen to remove that easy mass of near-static samples, and the increase in waypoint count is the single change that most improved the usefulness of the batch.

Figure 4.4 shows one resulting run, a fully randomized draw from the second half of the curriculum. The four cylinders move continuously and out of phase with one another, the extensions sweep a large fraction of their travel, and the valve commands are the smooth piecewise-linear profiles that the waypoint interpolation produces rather than abrupt steps. This is the realistic, coordinated, gravity-loaded operation that requirement R1 asks for, and it is exactly the regime the inverse and reduced-order models are trained to be accurate in.

4.3.3 Execution and logging

Each run simulates 22 s of operation. The solver is the stiff implicit `ode23t` integrator in accelerator mode with relative and absolute tolerances of 10^{-2} and 10^{-3}; the loose tolerances are a deliberate throughput compromise, and their effect is bounded by the consistency checks of Section 4.8. Output is written on a fixed 0.05 s grid, giving 441 logged samples per run, and the batch is checkpointed to disk every hundred runs so that a failure late in a multi-day batch does not cost the whole collection.

For every run the archive stores the time vector, the five valve commands, the four cylinder extensions, the four chamber pressure differences, the four rod forces, the five joint positions and velocities, the reference position and velocity signals, and, crucially, the complete parameter structure that produced the run. That last item is what makes the batch usable

for the latent-adaptation work: the privileged plant descriptor $\boldsymbol{\xi}$ of Section 2.6 is not something that has to be reconstructed later, it is recorded alongside the trajectory.

The cost is worth quoting, because it is the concrete form of the argument made in Section 2.1.3. A single 64 s open-loop run takes roughly 2000 s of wall time on the collection machine, a real-time factor of about 0.03. The Simscape plant runs some thirty times slower than the machine it models. No policy-optimisation loop that needs millions of environment steps can be built on that, which is precisely why the learned actuator surrogates exist.

4.4 Open-Loop Excitation Experiments

4.4.1 Why the closed-loop batch was not enough

The first forward-dynamics model was trained on the closed-loop material and scored a one-step R^2 near 0.999. It was also, as Section 5.3.2 documents, almost completely insensitive to its own valve input: zeroing that input changed the long-horizon rollout error by a few percent. The model had learned to continue a trajectory, not to respond to a command.

This is not a training failure and no amount of retraining on the same material would have fixed it. In closed loop the controller computes the command from the state, so the command is very nearly a deterministic function of the state history that the network already sees [Ljung 99]. The information needed to separate cause from correlation is simply not present in the data. Recovering it requires an experiment in which the command is generated independently of the state, which means opening the loop.

It is worth being clear that this is not a coverage problem in disguise. Figure 4.5 shows that the closed-loop batch is in fact varied in its control effort: pooled over a run, the valve spends about a quarter of the time saturated, roughly half in the mid-authority band, and the rest in low-command creep, so all three control regimes are represented. The difficulty is visible in the same figure. Plotting the valve command against the cylinder velocity it produced shows the two riding together along a band rather than filling the plane: in a feedback loop a fast motion is exactly when the controller commands a large valve opening, so command and state move as one. A model can read the command off the velocity, which is why it never has to learn what the command *causes*. Diverse control effort and identifiable control effort are not the same thing, and this batch has the first without the second.

4.4.2 Design of the excitation signal

The excitation run lasts 64 s and follows the fixed schedule in Figure 4.2(c). One second of startup is simulated and then discarded, because the plant is settling from its initial condition and those samples describe the initialisation rather than the dynamics. Four sequential slots follow, each giving one cylinder ten seconds of excitation followed by a half-second zero-command gap; the gaps make the boundaries between single-axis segments unambiguous. A twenty-second phase then drives all four channels at once, and a final second of zero command closes the run. Commands are limited to ± 0.90 of full authority.

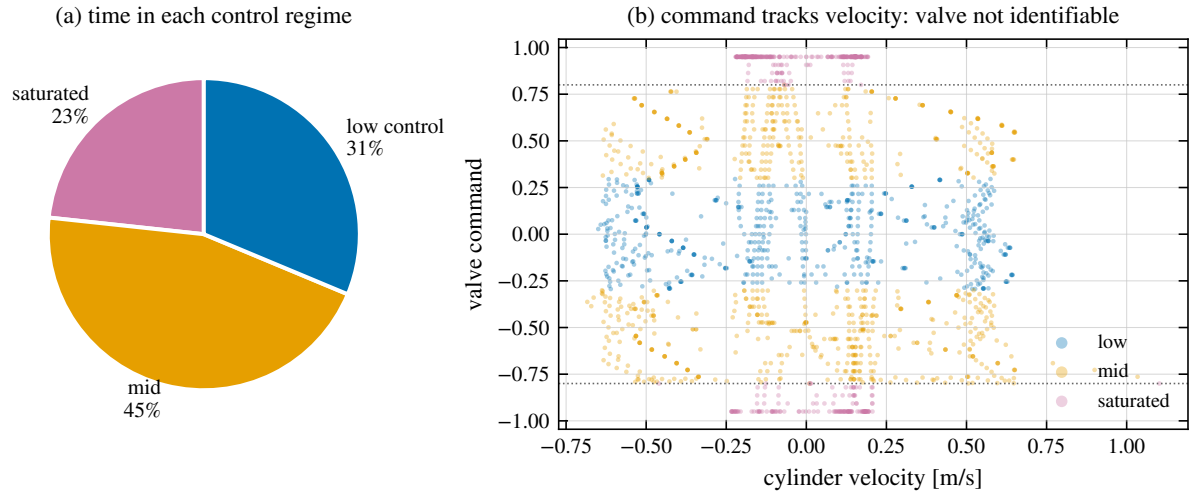


Figure 4.5: The closed-loop batch is diverse in control effort but not identifiable. (a) Fraction of time the valve spends saturated ($|u| \geq 0.8$), in the mid band ($0.3 \leq |u| < 0.8$), and in low-command creep ($|u| < 0.3$), pooled over a representative run; all three regimes are well represented. (b) The same samples plotted as valve command against the cylinder velocity they produced, coloured by regime. The command tracks the velocity along a band rather than covering the plane, so the valve is nearly a function of the state and cannot be separated from it by a model, whatever the control effort.

The most consequential design decision here is one that runs against the textbook reflex. A persistently exciting input for a linear system is usually a rapidly switching signal, and the first version of this excitation was built that way. It was persistently exciting in the formal sense and still produced poor data, because the cylinders reversed before they had travelled anywhere: the commanded energy went into repeatedly accelerating and decelerating the piston near one point of the stroke instead of moving it through the workspace. The present signal replaces fast switching with long full-authority plateaus during the sequential phase and a slow, high-authority component during the simultaneous phase, so that each cylinder actually traverses a substantial part of its stroke while its valve is open. The simultaneous phase additionally superimposes one slow near-square cycle across the full twenty seconds with a distinct medium-frequency component on each channel, and short raised-cosine ramps at entry and exit keep the transitions differentiable.

The lesson generalises, and it is one this thesis would offer to anyone identifying a hydraulic actuator: persistent excitation is a condition on the input signal, but the quantity that decides whether a *cylinder* is identifiable is how much of the stroke the input actually covers. The two are not the same, and optimising the first while ignoring the second produces data that passes the classical test and still cannot support a model.

4.4.3 Plant draws and acceptance

The same excitation is applied to four plant draws taken from the same parameter space as the closed-loop campaign: nominal, randomized hydraulics, randomized mechanics, and fully randomized. Holding the input fixed and varying only the plant is what makes the four runs directly comparable, since any difference in the response is attributable to the parameters.

Each run is checked for stroke coverage before it is accepted. The collector requires at least 10 % of the available stroke on cylinders 1–3 and 8 % on cylinder 4, warns below a target of 30 %, and records the achieved coverage per channel in the run summary. The runs retained here cover between 12 % and 33 % of the stroke depending on the cylinder, which is above the acceptance floor but below the target; cylinder 4 is the weakest, and improving its coverage is the clearest available next step for this campaign.

One earlier excitation export was rejected outright rather than repaired. Half of its runs turned out to contain duplicated state trajectories, so the nominally distinct plant draws were not distinct in the data, and the valve was again not causally identifiable. That export and its reports are not part of the maintained workflow. It is mentioned here because it is the reason the acceptance checks in Section 4.8 exist in their present form: they were written to make that particular failure impossible to miss a second time.

4.5 Combining Closed-Loop and Open-Loop Datasets

The two campaigns are complementary, and each learned model of Chapter 5 draws on them in a different proportion.

The inverse-dynamics model is trained entirely on closed-loop material, because its target is the command that a competent controller issued in a given state, and that is precisely what the closed-loop campaign records. The forward-dynamics model begins on closed-loop material and is then retrained on the strict open-loop export, with the closed-loop data retained as a replay term at a weight of 0.25 rather than discarded. That weighting is not cosmetic: ablation showed that dropping the replay term costs state coverage and extension accuracy, because four excitation runs cannot span the poses that a thousand operating runs visit. The open-loop data supplies causality, the closed-loop data supplies coverage, and the model needs both.

Figure 4.6 makes this complementarity concrete by comparing, per cylinder, where the two campaigns place the extension. The closed-loop distribution is broad and smoothly filled, because a thousand tracking runs visit the workspace densely; the open-loop distribution is concentrated into a few narrow bands, because the excitation holds each cylinder near a plateau for seconds at a time. The open-loop bands are where the valve is genuinely identifiable, and the closed-loop mass is where the model has to be accurate at deployment. Neither campaign covers both, which is why the forward model is trained on their union rather than on either alone.

The reduced-order model consumes the closed-loop material in a joined form. The inverse and forward exports are aligned on exact run identifiers, time indices, and sample times, and the join is accepted only when the sixteen-dimensional cylinder state agrees between them sample for sample. The resulting command-conditioned dataset contains 452 400 transitions at 0.05 s drawn from 1 040 aligned runs. The logged valve command is carried through this join, but only as a supervision target for the model's internal valve head; it is never a runtime input, which is the property that lets the ROM be driven by desired cylinder motion alone (Section 5.4).

The thousand-run randomized curriculum described in Section 4.3 occupies a different position in the argument, and I want to be exact about it because it is easy to overstate. The inverse, forward, and reduced-order models reported in Chapter 5 were trained on the

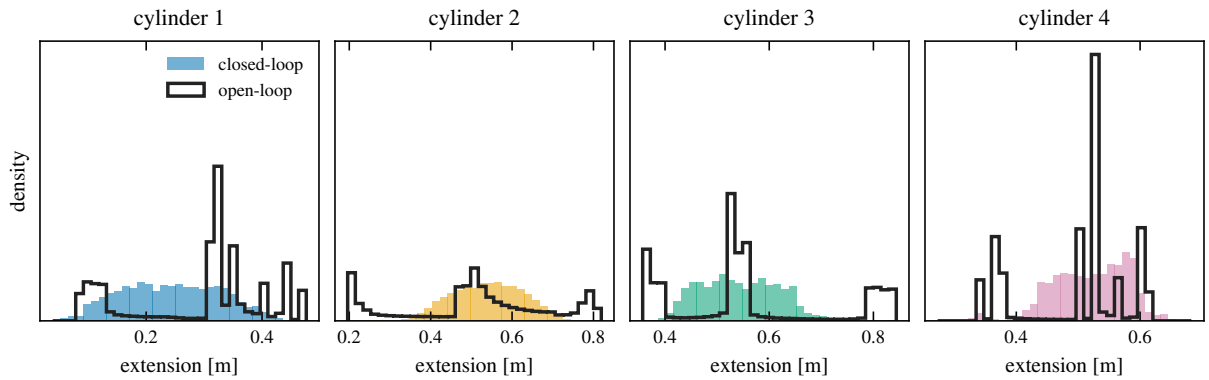


Figure 4.6: Extension coverage of the two campaigns, per cylinder, pooled over runs (closed-loop: filled, 120 curriculum runs; open-loop: outline, the four strict-excitation runs). The closed-loop data fills the reachable range smoothly, giving the pose coverage the models need at deployment; the open-loop data concentrates into the plateaus where the valve is causally identifiable. The two are complementary, and the forward model of Chapter 5 is trained on their union.

earlier aligned export, which is a nominal-plant collection. The randomized curriculum was collected to support the robust and adaptive configurations, PPO1-B, PPO2-B, and PPO2-C, and in particular to provide the per-run parameter vectors that the teacher encoder of Section 5.5 requires. It is the substrate for that work rather than a dataset whose results are already reported here.

4.6 Feature and Target Construction

All three learned actuator models take cylinder-space features, in the ordering and notation fixed in Section 5.1. Their input contracts differ because their questions differ. The inverse model receives the measured cylinder state, the logged rod forces, the desired increments, and the previously applied command, twenty-eight numbers in all. The forward model receives the applied valve command and the measured state, twenty numbers. The reduced-order model receives a six-sample history of state, previously predicted valve, and desired command, twenty-four numbers per step.

Three conventions apply throughout, and they are the same ones the models enforce at inference time.

Increment targets.

Every dynamic target is a within-run one-step increment, $\Delta \boldsymbol{x}_k = \boldsymbol{x}_{k+1} - \boldsymbol{x}_k$, never an absolute state. This removes the large static component of the signals and, more importantly, makes the trivial persistence solution an explicit baseline that a model can be measured against rather than a hiding place it can settle into.

Causal alignment.

Features at step k contain only quantities available at step k. The extraction stage verifies this by recomputing every logged target from the within-run difference of the states and requiring exact agreement, a check reported in Section 4.8.

Time-base conversion.

The open-loop campaign is logged at 0.01 s but the models run at 0.05 s, so five consecutive raw samples are combined into a single transition. The valve input attributed to that transition is the command held at the start of the window, which is the causally correct choice; the mean command over the window is also recorded, but only for analysis. Cylinder extension velocity is not logged directly in the closed-loop archive and is obtained by differentiating extension on the output grid, which is consistent with the trapezoidal identity that the strict data satisfies to $R^2 = 0.994$–0.9997 (Equation 5.10).

4.7 Preprocessing and Data Splits

Splitting is done at the level of complete runs, never of individual samples. A random sample-level split would place transitions from the same trajectory on both sides of the boundary and report a validation score that mostly measures interpolation within a run. For the command-conditioned dataset the 1 040 runs are divided into 936 training, 52 validation, and 52 test runs, giving 407 160, 22 620, and 22 620 transitions respectively, with no run identifier appearing in more than one split.

The strict open-loop export needs a different treatment because it contains only four runs, and holding one out would remove an entire plant draw from training. Instead each run is divided into twenty temporal blocks, every fifth block is assigned to validation, and a five-sample guard interval is discarded at each block boundary so that no 0.05 s transition window spans the split. A simple last-twenty-percent split was tried first and rejected: it changed the spread of the targets between training and validation by a factor of 1.3 to 2.3, because the final fifth of the run is the simultaneous excitation phase and is dynamically unlike everything before it. That split would have measured extrapolation to a different regime and called it generalisation.

Normalization statistics are computed on the training split only and stored inside each checkpoint, so that a model always normalizes with the statistics it was fitted with. Mixing statistics between checkpoints silently invalidates any comparison, and the loader enforces the stored contract rather than recomputing it.

Finally, samples are carried with an explicit validity flag rather than being silently dropped. In the strict export 93.7 % of samples are marked valid; the remainder are the excluded startup and shutdown seconds and the zero-command gaps. Keeping them in the archive with a flag, instead of deleting them, means the time base stays contiguous and the within-run difference checks remain meaningful.

4.8 Dataset Diagnostics

No dataset in this thesis is used before it passes an explicit audit, for the reason given at the end of Section 4.4. The audit of the strict open-loop export is reproduced in Table 4.3.

The two rank figures are the ones that speak directly to requirement R2. The instantaneous command matrix has rank four, so no channel is a linear combination of the others, and the lagged regressor matrix built from the command history has rank eighty, so the excitation is persistently exciting to the order the models need. The condition number of that lagged matrix is about 4.6×10^3, which is finite and workable rather than comfortable; it is a

Table 4.3: Audit of the strict open-loop excitation export. All checks are computed by the extraction stage and stored with the data; the export is accepted only if every one of them passes.

Check	Result	Interpretation
Rows, runs, columns	25 600 / 4 / 49	four complete runs
Median sample time	0.01 s	as commanded
Maximum sample-time error	9.1×10^{-15} s	numerically exact grid
Maximum Δextension error	0	targets match state differences
Maximum Δvelocity error	0	targets match state differences
Maximum $\Delta(\Delta p)$ error	0 Pa	targets match state differences
Non-finite values	0	no solver escapes
Valid fraction	0.937	startup, gaps, shutdown excluded
Command rank	4	all four channels independent
Lagged excitation rank	80	history matrix full rank
Valve saturation	8.5 % per channel	limits reached but not dominant
Stroke coverage	12 % – 33 %	above floor, below target

consequence of using long plateaus, and it is the price paid for the stroke coverage those plateaus buy.

The exactness of the three increment checks deserves a word, since a maximum error of exactly zero invites suspicion. It is expected here: the targets are computed from the logged states by the same within-run difference that the check recomputes, so the check verifies that the extraction stage did what it claims, not that the simulation is accurate. It catches misalignment, off-by-one indexing, and rows that have been reordered or concatenated across run boundaries, which are the failure modes that actually occur. Physical fidelity is a separate question, addressed by the validation study in Section 3.4.3.

Three limitations are carried forward honestly. The base fluid stiffness does not vary across the randomized batch, for the clamp reason set out in Section 4.2; the mechanical randomization, the discharge coefficient, the seal wear, the relief setting, and the dead-volume capacitance all vary as intended and are confirmed in the stored run descriptors, but no result in this thesis may be read as adaptation to bulk modulus specifically. Stroke coverage in the open-loop campaign reaches only 12 % on the head-pivot cylinder, which is the cylinder the strict forward model predicts least well, and the two facts are unlikely to be unrelated. And the rod-force channel is recorded but fails a piston-area consistency check on this export, which is why it carries zero supervised weight in the reduced-order model (Section 5.4) and why no result in this thesis rests on it.

With the data described and audited, the next chapter turns to what was learned from it: the inverse and forward models that form the PPO1 actuator path, and the command-conditioned reduced-order model that replaces them in PPO2.

5. Learned Models and Online Adaptation

The previous chapter established how the Simscape datasets were generated and why both closed-loop operation and open-loop valve excitation were required. This chapter develops the learned actuator representations that consume those datasets: the inverse-dynamics (ID) and forward-dynamics (FD) networks that form the PPO1 actuator path, and the command-conditioned reduced-order model (ROM) that replaces them in the PPO2 path. Each model is presented with its exact input–output contract, architecture, training objective, and offline validation evidence, because a recurring result of this work is that standard supervised scores alone were not sufficient to accept any of these models: a forward model can achieve near-perfect one-step R^2 while having learned almost nothing about valve authority (Section 5.3). All models are trained on cylinder-space signals only; the mapping between cylinder space and task space is handled by the kinematic layers of Chapter 3 and the control integration is described in Chapter 6.

5.1 Modeling Overview and Common Notation

5.1.1 Cylinder-space signals

All learned models in this thesis operate on the four hydraulic cylinders of the MULAG arm, indexed $i \in \{1, \ldots, 4\}$ in the fixed order slew cylinder (*Drehzapfen*), boom cylinder I (*Ausleger I*), boom cylinder II (*Ausleger II*), and head pivot cylinder (*Messerkopf-Schwenk*). The passive fifth joint is not actuated and does not appear in any model input. At the learned-model sample time $T = 0.05\,\mathrm{s}$, the discrete time index k refers to the cylinder-space state

$$\boldsymbol{x}_k \in \mathbb{R}^4, \qquad \dot{\boldsymbol{x}}_k \in \mathbb{R}^4, \qquad \Delta\boldsymbol{p}_k \in \mathbb{R}^4, \qquad \boldsymbol{F}_k \in \mathbb{R}^4, \tag{5.1}$$

where $\boldsymbol{x}$ collects the cylinder extensions in meters, $\dot{\boldsymbol{x}}$ the extension velocities, $\Delta\boldsymbol{p}$ the chamber pressure differences, and $\boldsymbol{F}$ the logged rod forces. The valve command vector $\boldsymbol{u}_k \in [-1, 1]^4$ is the normalized spool command of the four proportional valves. Desired

Table 5.1: Cylinder-space notation used by all learned models.

Symbol	Dimension	Meaning
T	–	learned-model sample time, 0.05 s
$\boldsymbol{x}_k$, $\dot{\boldsymbol{x}}_k$	$\mathbb{R}^4$	cylinder extension [m] and extension velocity [m/s]
$\Delta\boldsymbol{p}_k$	$\mathbb{R}^4$	chamber pressure difference [Pa]
$\boldsymbol{F}_k$	$\mathbb{R}^4$	logged rod force [N]
$\boldsymbol{u}_k$	$[-1,1]^4$	normalized proportional-valve command
$\boldsymbol{c}_k = [\Delta\boldsymbol{x}_k^*, \Delta\dot{\boldsymbol{x}}_k^*]$	$\mathbb{R}^8$	desired extension and velocity increments
$\Delta(\cdot)_k = (\cdot)_{k+1} - (\cdot)_k$	–	within-run one-step increment
$\hat{(\cdot)}$	–	model prediction
$\boldsymbol{h}_k$	$\mathbb{R}^{128}$	recurrent latent state of the ROM encoder

quantities carry a star; the desired cylinder command produced by the task controller is the pair

$$\boldsymbol{c}_k = \left[\Delta\boldsymbol{x}_k^*,\ \Delta\dot{\boldsymbol{x}}_k^*\right] \in \mathbb{R}^8, \tag{5.2}$$

i.e. the extension increment and the extension-velocity increment that the controller wants to realize over the next sample. Predictions of a model carry a hat. Table 5.1 summarizes the notation used throughout this chapter.

5.1.2 Two actuator representations, one contract

Figure 5.1 shows the two learned actuator paths side by side. In the PPO1 path, actuation is represented by two separate networks: the ID network maps the current hydraulic state and a desired increment to a valve command, and the FD network maps a valve command and the current state to the predicted state increment. Their composition $f_{\mathrm{FD}} \circ f_{\mathrm{ID}}$ predicts what a commanded increment will actually do to the cylinders. In the PPO2 path, one command-conditioned ROM replaces this composition: it receives the same desired command $\boldsymbol{c}_k$, predicts the valve internally, and propagates pressure, velocity, and extension through an explicit, physically structured update (Section 5.4). Both paths terminate in the same bounded PhysX cylinder position target inside the IsaacLab environment [Mittal 23], and both are driven by the same damped-least-squares task controller and residual PPO policy [Schulman 17, Johannink 19] described in Chapter 6. The experimental variable between PPO1 and PPO2 is therefore the actuator representation alone.

5.1.3 Common conventions

Three conventions apply to every model in this chapter.

Increment targets.

All dynamic targets are within-run one-step increments, $\Delta\boldsymbol{x}_k = \boldsymbol{x}_{k+1} - \boldsymbol{x}_k$, never absolute states. This removes the dominant static component of the signals and makes the persistence solution ($\Delta\hat{\boldsymbol{x}} = \boldsymbol{0}$) an explicit, testable baseline rather than a hidden local minimum.

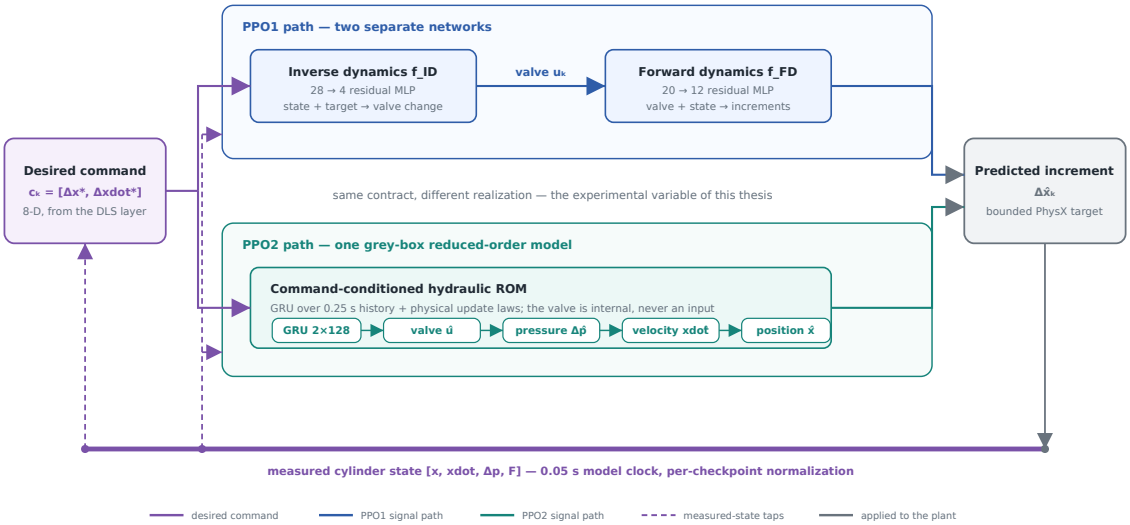


Figure 5.1: The two learned actuator representations as parallel lanes around one shared contract. Both consume the same desired cylinder command and the same measured cylinder state from the feedback bus, and both produce the predicted increment that becomes the bounded PhysX target. The PPO1 lane composes two separately trained networks; the PPO2 lane realizes the command inside a single grey-box model whose valve signal is internal. Training-time data flow is described in Chapter 4.

Per-checkpoint normalization.

Every checkpoint stores the mean $\boldsymbol{\mu}$ and standard deviation $\boldsymbol{\sigma}$ of its own training inputs and targets, and inference normalizes each feature as

$$\tilde{\xi} = \frac{\xi - \mu_\xi}{\sigma_\xi}. \tag{5.3}$$

Statistics are never shared between checkpoints; mixing the statistics of two checkpoints silently invalidates any comparison between them, so the loader enforces the stored contract.

Causal timing.

Model inputs at step k contain only quantities available at step k (including the previously applied command), and targets are the measured transition to step $k+1$. During dataset construction this alignment is checked explicitly against the within-run differences of the logged states (Chapter 4).

Gate-based acceptance.

Finally, no model in this thesis is promoted on a supervised score. Every candidate passes the fixed gate sequence of Figure 5.2: offline accuracy, input-ablation authority, in-simulator cylinder gates, and the task-space tracking gate, in that order. Each "no" exit names the repair that experience showed to be the right one, and the promoted checkpoints are recorded with their hashes for provenance. The sections that follow report, for each model, exactly where in this ladder it initially failed and what fixed it.

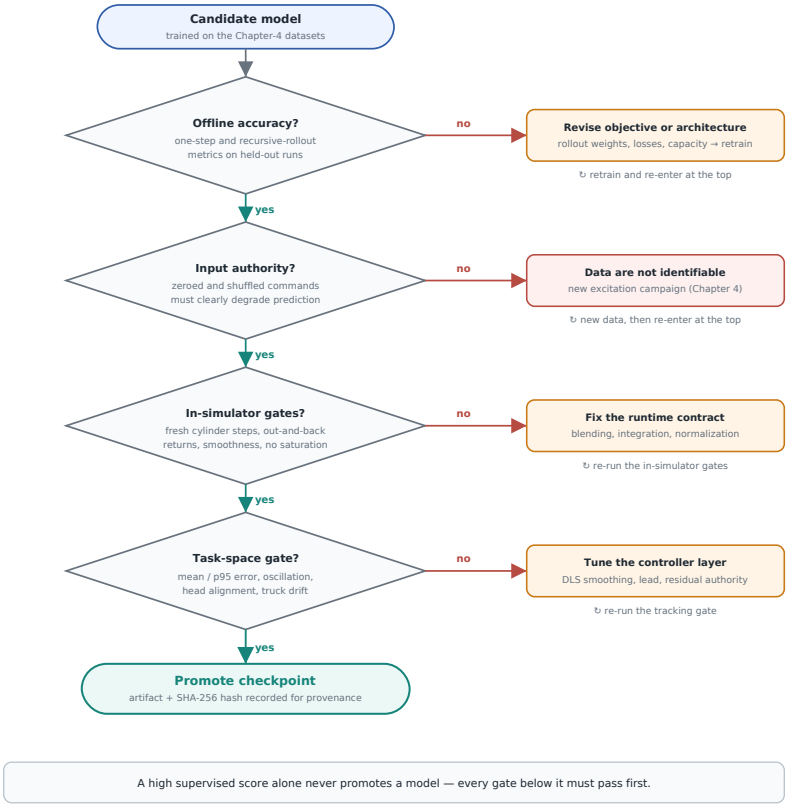


Figure 5.2: The acceptance ladder that every learned model in this thesis must climb before deployment. A candidate that fails a gate exits right into the corresponding repair loop and re-enters from the top; only a candidate that passes all four gates is promoted, and the promoted artifact is recorded together with its SHA-256 hash.

5.1.4 Data provenance in brief

Two dataset families from Chapter 4 are consumed here. The *closed-loop* family consists of the aligned Simscape exports [The MathWorks, Inc. 24] used for supervised ID and FD training and, after exact run-and-timestamp alignment, for the command-ROM dataset: 452 400 transitions at $T = 0.05\,\mathrm{s}$ from 1 040 runs, split by complete runs into 936 training, 52 validation, and 52 test runs (407 160 / 22 620 / 22 620 samples). The *open-loop* family is the strict valve-excitation export used to repair the FD model: four 64-s runs sampled at 0.01 s, in which the same rank-4 multi-input open-loop excitation is applied to four distinct plant draws (nominal, randomized hydraulics, randomized mechanics, both). Its properties and the reasons it was necessary are analyzed in Section 5.3.

5.2 Inverse Dynamics Model

5.2.1 Input–output contract

The ID network answers the question the low-level controller actually asks: *given the current hydraulic state, which valve command realizes a desired cylinder increment?* Its

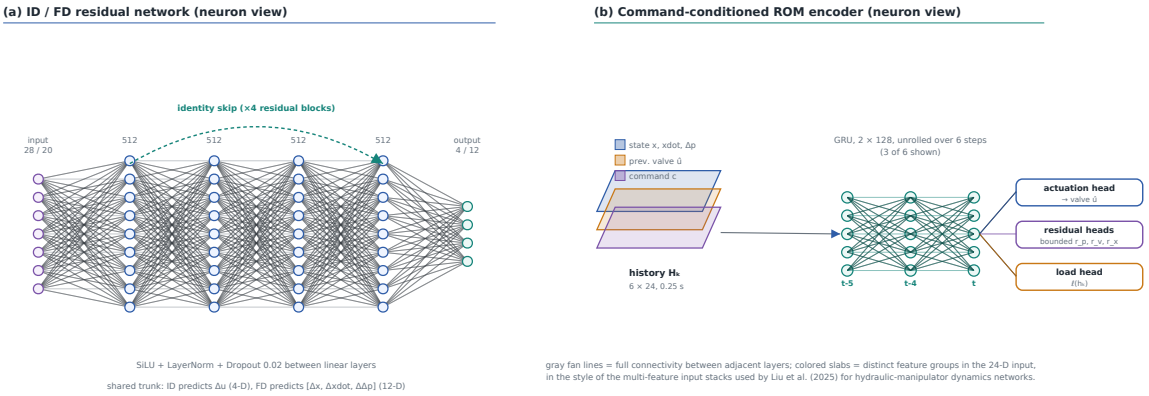


Figure 5.3: The two network families used by the learned actuator models. (a) The residual MLP trunk shared by the ID and FD networks: an input projection to width 512, four pre-activation residual blocks (Linear–LayerNorm–SiLU–Dropout–Linear with an identity skip), and a linear output head; only the input and output widths differ between ID and FD. (b) The ROM's recurrent encoder, unrolled over its six history steps: each step consumes the 24-D feature vector, the hidden state carries pressure and command transients across time, and the final latent $\boldsymbol{h}_k$ feeds the actuation, residual, and load heads.

contract is

$$\Delta \boldsymbol{u}_k = f_{\mathrm{ID}}\Big(\boldsymbol{x}_k,\, \dot{\boldsymbol{x}}_k,\, \Delta \boldsymbol{p}_k,\, \boldsymbol{F}_k,\, \Delta \boldsymbol{x}_k^*,\, \Delta \dot{\boldsymbol{x}}_k^*,\, \boldsymbol{u}_{k-1}\Big) \in \mathbb{R}^4, \tag{5.4}$$

a 28-dimensional input mapped to a valve-command *change*. The deployed command is the accumulated

$$\boldsymbol{u}_k = \mathrm{clip}\Big(\boldsymbol{u}_{k-1} + \Delta \boldsymbol{u}_k,\; -\boldsymbol{u}_{\max},\, \boldsymbol{u}_{\max}\Big), \qquad \boldsymbol{u}_{\max} = [0.95,\, 0.95,\, 0.8,\, 0.8], \tag{5.5}$$

where the per-axis limits are stored in the checkpoint. Predicting the command change rather than the absolute command conditions the problem on the previous operating point of the valve, which matters because proportional valves are direction- and history-dependent around their dead zone (Section 2.1.2). The desired increments $\Delta \boldsymbol{x}^*, \Delta \dot{\boldsymbol{x}}^*$ in the training data are the *achieved* transitions of the logged Simscape runs, used as a proxy for desired motion because no independent cylinder-space reference was logged; the consequences of this proxy are examined below.

5.2.2 Architecture and supervised training

f_{ID} is a multilayer perceptron with an input projection to width 512 followed by four residual blocks [He 16] with SiLU activations [Elfwing 18], layer normalization [Ba 16], and dropout of 0.02 [Srivastava 14], implemented in PyTorch [Paszke 19]; Figure 5.3(a) shows the trunk, which the FD network shares, and Figure 5.3(b) the recurrent encoder that the ROM uses instead (Section 5.4).

Training minimized the smooth-L_1 (Huber) loss [Huber 64] with transition point $\beta = 0.5$ on normalized targets using Adam [Kingma 15] with learning rate 10^{-3}, weight decay

10^{-4}, batch size 8 192, gradient-norm clipping at 10, and early stopping with patience 45; samples in valve saturation were up-weighted by a factor 1.25 so that the rare saturated regime is not ignored. The promoted checkpoint (seed 42) was selected at epoch 120 of at most 300 with a validation loss of 0.0336.

The supervised result was, on its face, excellent: per-valve validation R^2 of 0.9996, 0.9973, 0.9981, and 0.9974. It must be stated clearly what this number does and does not mean. It means the network reproduces the logged valve commands from the logged states and achieved transitions. It does not mean the composite map $f_{\mathrm{FD}} \circ f_{\mathrm{ID}}$ has authority over the plant: since the achieved transitions in closed-loop data are themselves produced by the logged commands, a model can score highly by exploiting this correlation without ever being asked to *cause* a transition that was not already in the data (Section 2.4.2).

5.2.3 Paired control fine-tuning

This deficiency became measurable when the ID and FD networks were composed. For control, the relevant object is the *paired differential response*: the difference between what the FD predicts under the ID's command for the actual target and under the ID's command for a zero target,

$$\begin{aligned} \boldsymbol{u}_k^{\mathrm{tgt}} &= f_{\mathrm{ID}}(\cdot,\, \boldsymbol{c}_k,\, \boldsymbol{u}_{k-1}), \qquad \boldsymbol{u}_k^{0} = f_{\mathrm{ID}}(\cdot,\, \mathbf{0},\, \boldsymbol{u}_{k-1}), \\ \Delta\hat{\boldsymbol{x}}_k^{\mathrm{pair}} &= f_{\mathrm{FD}}\Big(\boldsymbol{u}_k^{\mathrm{tgt}}, \cdot\Big) - f_{\mathrm{FD}}\Big(\boldsymbol{u}_k^{0}, \cdot\Big). \end{aligned} \tag{5.6}$$

If the pipeline is to be useful, $\Delta\hat{\boldsymbol{x}}_k^{\mathrm{pair}}$ must track the requested $\Delta\boldsymbol{x}_k^{*}$. Before fine-tuning it did not: the normalized control error of the composite was 1.00 — exactly the score of predicting nothing — with per-axis regression slopes of at most 0.004 (Table 5.2). Both networks were therefore fine-tuned jointly on the control objective

$$\mathcal{L}_{\mathrm{ctl}} = \left\| \Delta\hat{\boldsymbol{x}}^{\mathrm{pair}} - \Delta\boldsymbol{x}^{*} \right\|_{\mathrm{norm}}^{2} \;+\; \lambda_{\mathrm{ID}}\, \mathcal{L}_{\mathrm{ID}} \;+\; \lambda_{\mathrm{FD}}\, \mathcal{L}_{\mathrm{FD}}, \tag{5.7}$$

where the supervised losses $\mathcal{L}_{\mathrm{ID}}$ and $\mathcal{L}_{\mathrm{FD}}$ are retained to limit regression from the original data contracts. Crucially, the target blocks $(\Delta\boldsymbol{x}^{*}, \Delta\dot{\boldsymbol{x}}^{*})$ were shuffled across valid states during this stage, so the networks could no longer succeed by reconstructing the logged trajectory; they had to respond to targets that the logged controller never issued in that state.

After 30 epochs of paired tuning the composite control MSE dropped to 0.109 and all four axes reached slopes above 0.8 and correlations above 0.92. The resulting ID checkpoint (`best_id_cylinder_control.pt`) is the ID model deployed in all PPO1 experiments. The fine-tuned FD from this stage, in contrast, was accepted only as an intermediate: its remaining weakness under genuinely independent valve excitation is the subject of the next section.

5.3 Forward Dynamics Model

5.3.1 Input–output contract and supervised baseline

The FD network predicts the one-step cylinder response to a valve command,

$$\Big[\Delta\hat{\boldsymbol{x}}_k,\; \Delta\hat{\dot{\boldsymbol{x}}}_k,\; \Delta\widehat{\Delta\boldsymbol{p}}_k\Big] = f_{\mathrm{FD}}\Big(\boldsymbol{u}_k,\, \boldsymbol{x}_k,\, \dot{\boldsymbol{x}}_k,\, \Delta\boldsymbol{p}_k,\, \boldsymbol{F}_k\Big) \in \mathbb{R}^{12}, \tag{5.8}$$

Table 5.2: Effect of paired control fine-tuning on the composite $f_{\mathrm{FD}} \circ f_{\mathrm{ID}}$ differential response (5.6), evaluated per cylinder axis on validation data. Before fine-tuning, the composite had essentially no control authority despite near-perfect supervised scores of both networks; the composite normalized control MSE fell from 1.0011 to 0.1086.

	Regression slope		Correlation	
Cylinder axis	before	after	before	after
1 (slew)	0.003	0.818	0.135	0.920
2 (boom I)	0.002	0.928	0.090	0.967
3 (boom II)	0.001	0.849	0.037	0.933
4 (head pivot)	0.004	0.912	0.249	0.958

a 20-dimensional input mapped to the three groups of state increments over $T = 0.05\,\mathrm{s}$. The architecture is the same residual MLP trunk as the ID network (width 512, four residual blocks, SiLU, layer normalization, dropout 0.02). Supervised pretraining on the closed-loop Simscape export used normalized MSE on all three output groups, Adam with learning rate 10^{-4}, weight decay 10^{-4}, batch size 8 192, and an auxiliary recursive rollout loss (weight 0.25, horizons of 2, 5, 10, and 20 steps), with the checkpoint selected by validation rollout loss [Nagabandi 18]. The selected model (epoch 291) reached grouped one-step R^2 near 0.999 for the extension increments.

5.3.2 Why the supervised score was not believed

A model intended to give a policy gradient information about actuation must respond to the *valve*, not merely to the state history. This property was tested directly with an input ablation: zeroing the valve input of the pretrained FD barely changed its long-horizon rollout error (valve-zero error ratio ≈ 0.94 at a 0.5 s horizon; Figure 5.5, dashed orange versus solid orange). The model had learned to extrapolate the smooth kinematics of closed-loop operation — effectively a sophisticated persistence model — because in feedback data the controller makes valve command and state history nearly redundant, the identifiability trap of Section 2.4.2. Ablation studies on the source MAT exports (Chapter 4) confirmed that the valve signal was not causally identifiable from the closed-loop distribution at all. No amount of retraining on the same distribution could fix this; the data had to change.

5.3.3 Strict open-loop excitation dataset

The repair dataset is the strict open-loop excitation export shown in Figure 5.4. All four valves receive a rank-4 multi-input open-loop sequence built from long, full-authority command plateaus: the first four segments excite one cylinder at a time, and the final segment drives all four channels simultaneously (Section 4.4). The same excitation is applied to four distinct plant draws — nominal, randomized hydraulics, randomized mechanics, and both randomized — giving four 64-s runs at 0.01 s (6 400 samples each, 49 logged columns). Dataset validation established the properties that the closed-loop export lacked: all four state trajectories are distinct; every logged target increment agrees with the within-run difference $\boldsymbol{x}_{k+1} - \boldsymbol{x}_k$; valve history alone predicts the pressure response with $R^2 \approx 0.80$–0.85; the valve regressors have variance-inflation factors of only 7–10;

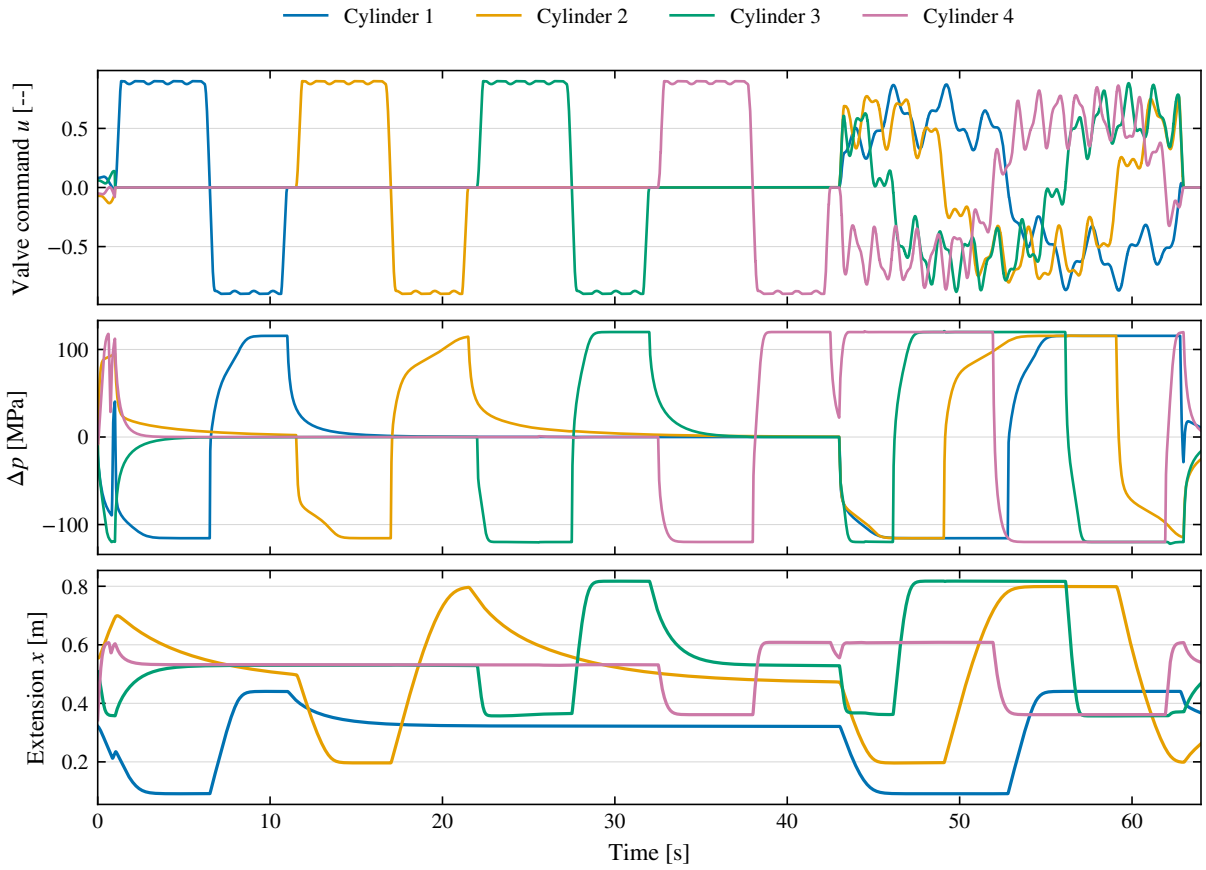


Figure 5.4: One run of the strict open-loop excitation dataset (nominal plant draw, 0.01 s sampling). Top: the rank-4 open-loop valve commands; the channels are excited individually in the first four segments and simultaneously in the final segment. Middle: resulting chamber pressure differences, including relief-limit episodes. Bottom: cylinder extension responses. The same excitation is repeated on three further randomized plant draws.

and pressure-relief saturation occupies 9–22 % of the samples, so the saturated regime is represented but not dominant. An earlier excitation export was rejected outright because half of its runs duplicated state trajectories and the valve was again not causally identifiable — a reminder that excitation data must itself be audited, not assumed informative [Isermann 11].

For FD training, five consecutive 0.01 s samples are combined into one 0.05 s transition, with the valve input taken as the command held at the beginning of the window. Each run is divided into 20 temporal blocks with every fifth block assigned to validation and five-sample guard intervals removed at each boundary, so no transition window crosses a split boundary while train and validation samples still cover comparable pose and energy regimes. A plain last-20 % split was tried and rejected because it changed the target spread between train and validation by a factor of 1.3–2.3 and would therefore have tested extrapolation to a different operating regime rather than generalization within one.

5.3.4 Strict retraining objective

Starting from the paired-control FD checkpoint, the promoted retraining run (150 epochs, Adam, learning rate 3×10^{-4}) minimized

$$\mathcal{L}_{\mathrm{FD}} = \mathcal{L}^{\mathrm{strict}}_{\mathrm{Huber}} + 0.25\,\mathcal{L}^{\mathrm{MAT}}_{\mathrm{Huber}} + 2.0\,\mathcal{L}^{(10)}_{\mathrm{roll}} + 10.0\,\mathcal{L}_{\mathrm{kin}}, \tag{5.9}$$

combining the one-step Huber loss on the strict data, a replay term on the original closed-loop data (weight 0.25, retained because ablation showed it improves state coverage and extension accuracy), a 10-step (0.5 s) recursive rollout loss, and a kinematic-consistency term enforcing the trapezoidal identity

$$\Delta \boldsymbol{x}_k = T\,\dot{\boldsymbol{x}}_k + \tfrac{1}{2}\,T\,\Delta\dot{\boldsymbol{x}}_k, \tag{5.10}$$

which holds in the measured strict data with $R^2 = 0.994$–0.9997 and ties the predicted extension and velocity increments to each other. The Huber loss is used because rare relief events otherwise dominate a squared-error objective. One further, initially attractive regularizer was evaluated and deliberately rejected: anchoring the model to a constant valve Jacobian. The measured valve authority varies by a factor of 15–75 across pressure quartiles and can change sign, so pinning a single global gain produced a pleasing mean response while collapsing one-step accuracy. This is consistent with the strongly operating-point-dependent valve–flow–pressure characteristics of hydraulic servo-systems discussed in Section 2.1.2, and it motivates the pressure-dependent structure adopted in the ROM (Section 5.4).

5.3.5 Validation: valve authority restored

Figure 5.5 and Table 5.3 summarize the outcome. The strict FD reduces the 0.5 s extension rollout RMSE from 75.2 mm to 12.4 mm and the 1.0 s RMSE from 157.9 mm to 27.3 mm, and — the decisive criterion — its valve-zero error ratio rises from 0.94 to 1.81 at 0.5 s: removing the valve input now roughly doubles the rollout error, which is exactly the sensitivity a control-relevant forward model must exhibit. Per-axis one-step extension-increment R^2 improved from the range $[-5.7, 0.60]$ to $[0.76, 0.98]$. Two honest limitations remain. First, at very short horizons (≤ 0.25 s) the zero-order hold baseline is still competitive, because extension simply does not move far in 0.05 s; the learned model's value appears at the horizons over which a target increment must be committed. Second, the velocity-increment outputs remain poorly predicted (near-zero or negative R^2 on two axes, dominated by relief transients that a memoryless 0.05 s model cannot resolve). The deployed controller therefore integrates only $\Delta\hat{\boldsymbol{x}}$, the accurately modeled and directly validated quantity, and treats the velocity output as diagnostic.

For completeness, an intermediate rank-preserving fine-tuning stage between the supervised and paired stages regularized the valve-to-extension differential map with a finite-difference Jacobian penalty; the resulting checkpoint has a full-rank (rank-4) valve Jacobian with condition number ≈ 2.3 and is retained as provenance, but the strict FD supersedes it as the active model. The strict FD (`best_fd_cylinder_strict.pt`) together with the paired-control ID constitutes the learned actuator pair of every PPO1 experiment in Chapter 7.

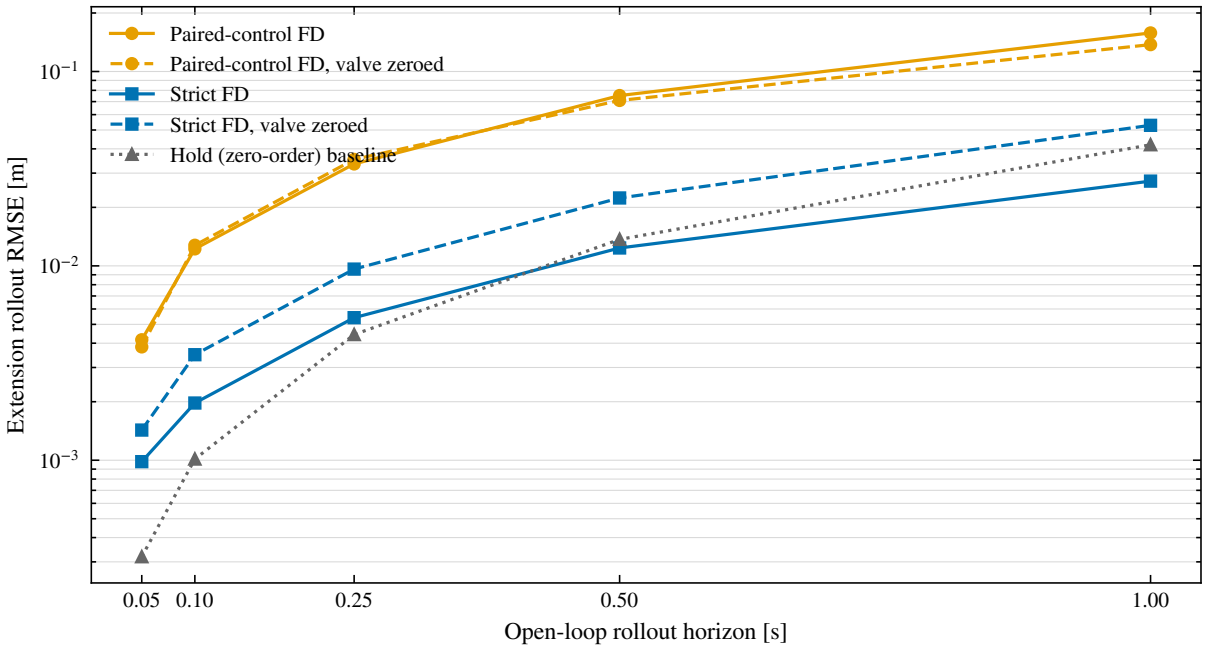


Figure 5.5: Open-loop extension rollout RMSE on held-out strict-excitation data as a function of rollout horizon, before (orange) and after (blue) strict retraining. Dashed curves show the same model with the valve input zeroed; the gap between solid and dashed is the model's usable valve authority. The paired-control FD is insensitive to its valve input, whereas the strict FD degrades by a factor of 1.8–1.9 when the valve is removed and outperforms the zero-order hold baseline (gray) beyond 0.5 s.

5.4 Reduced-Order Model

5.4.1 Motivation and design position

The PPO1 actuator path composes two separately trained networks and, as the previous sections showed, needed three repair stages before the composition had authority. The PPO2 path replaces the composition with a single *command-conditioned reduced-order model* that is trained end-to-end on exactly the quantity the controller consumes: the response of the cylinders to a desired cylinder command. Two design decisions distinguish the ROM from a generic sequence model. First, it is *grey-box*: pressure, force, velocity, and extension are propagated through explicit physical update equations with trainable effective parameters, and neural components only supply what the physics cannot —

Table 5.3: Strict-excitation validation of the FD model before and after strict retraining. The valve-zero error ratio is the rollout RMSE with zeroed valve input divided by the RMSE with the true valve input at a 0.5 s horizon; a ratio near 1 means the model ignores the valve.

Metric	Paired-control FD	Strict FD
Valve-zero error ratio at 0.5 s	≈ 0.94	1.81
Extension rollout RMSE at 0.5 s	75.2 mm	12.4 mm
Extension rollout RMSE at 1.0 s	157.9 mm	27.3 mm
One-step Δx R^2 (per-axis range)	-5.7 to 0.60	0.76 to 0.98

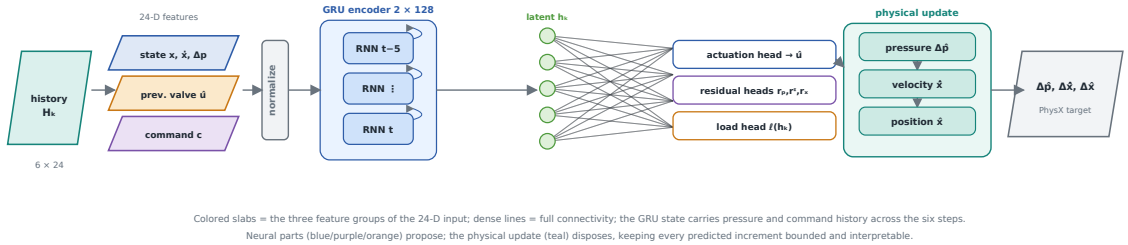


Figure 5.6: Structure of the command-conditioned reduced-order model as a data-flow pipeline. The 0.25 s history is split into its three feature groups (state, previously predicted valve, desired command), normalized, and passed through the recurrent GRU encoder whose hidden state carries pressure and command history across the six steps. The final latent feeds three fully connected heads: the actuation head realizes the command as an internal valve signal, the residual heads supply tanh-bounded corrections, and the load head the configuration load. The state is then propagated through the explicit pressure (5.13), velocity (5.14), and position (5.15) updates with trainable effective hydraulic parameters, so the neural parts propose and the physics disposes.

the valve realization, unmodeled load, and bounded residual corrections [Karniadakis 21, Lutter 19]. This follows the observation from Section 5.3 that valve authority is strongly pressure-dependent: encoding the orifice-flow–continuity structure [Merritt 67] makes that dependence architectural rather than something to be rediscovered from data. Second, it is *recurrent*: a GRU encoder [Cho 14] over 0.25 s of history replaces the memoryless state input of the FD, in the spirit of the learned actuator networks of Hwangbo et al. [Hwangbo 19], because hydraulic response depends on recent pressure and command history that a single sample cannot represent.

5.4.2 Input–output contract

The ROM consumes a history of $L = 6$ samples (0.25 s at $T = 0.05$ s) of the 24-dimensional feature vector

$$\boldsymbol{z}_k = \left[\boldsymbol{x}_k,\ \dot{\boldsymbol{x}}_k,\ \Delta\boldsymbol{p}_k,\ \hat{\boldsymbol{u}}_{k-1},\ \boldsymbol{c}_k\right] \in \mathbb{R}^{24}, \tag{5.11}$$

i.e. the cylinder state, the *previously predicted* valve command fed back as recurrent state, and the current desired command $\boldsymbol{c}_k \in \mathbb{R}^8$ of (5.2). The actual valve command appears only as a training target for the internal valve head; at runtime no valve signal enters the model. From $\mathcal{H}_k = (\boldsymbol{z}_{k-L+1}, \ldots, \boldsymbol{z}_k)$ the ROM predicts the internal valve realization and the full next cylinder state,

$$\left(\hat{\boldsymbol{u}}_k,\ \Delta\hat{\boldsymbol{p}}_k,\ \Delta\hat{\dot{\boldsymbol{x}}}_k,\ \Delta\hat{\boldsymbol{x}}_k\right) = f_{\mathrm{ROM}}\left(\mathcal{H}_k\right), \tag{5.12}$$

which jointly replaces the ID–FD composition of the PPO1 path.

5.4.3 Grey-box structure

Figure 5.6 shows the computation. The normalized history is encoded by a two-layer GRU of width 128 (dropout 0.05); the final latent $\boldsymbol{h}_k$ and the command $\boldsymbol{c}_k$ feed a small actuation head whose output passes through a learnable smooth dead-zone element —

per-cylinder trainable width and asymmetric gains, built from softplus and tanh primitives so it remains differentiable — reflecting the dead band and direction-dependent gain of proportional valves (Section 2.1.2). The resulting internal actuation $\hat{a}_i \in [-1, 1]$ drives an orifice-flow and continuity update per cylinder i,

$$q_i = \hat{a}_i \, k_{q,i} \sqrt{\tfrac{1}{2}\big(p_{s,i} - \operatorname{sgn}(\hat{a}_i)\, \Delta p_i\big)_+},$$
$$\Delta\hat{p}_{i,k+1} - \Delta p_{i,k} = \frac{4\beta_i}{V_i(x_i)}\big(q_i - A_i\dot{x}_i - k_{\ell,i}\,\Delta p_i\big)\,T \;+\; r_{p,i}, \tag{5.13}$$

with effective bulk modulus β_i, supply pressure $p_{s,i}$, flow gain $k_{q,i}$, leakage coefficient $k_{\ell,i}$, piston area A_i, and extension-dependent chamber volume $V_i(x_i) = V_{0,i} + A_i(x_i - x_{\mathrm{ref},i})$; the softplus-smoothed pressure margin $(\cdot)_+$ keeps the square root differentiable near the supply limit. The pressure then produces force, and force produces motion:

$$\hat{F}_i = \Delta\hat{p}_{i,k+1} A_i + r_{F,i}, \qquad \ddot{x}_i = \frac{\hat{F}_i - c_{v,i}\,\dot{x}_i - \ell_i(\boldsymbol{h}_k)}{m_i},$$
$$\dot{\hat{x}}_{i,k+1} = \dot{x}_{i,k} + \operatorname{sat}\big(T\,\ddot{x}_i + r_{v,i}\big), \tag{5.14}$$
$$\hat{x}_{i,k+1} = x_{i,k} + \tfrac{T}{2}\big(\dot{x}_{i,k} + \dot{\hat{x}}_{i,k+1}\big) + r_{x,i}, \tag{5.15}$$

where $c_{v,i}$ is a viscous friction coefficient, m_i an effective modal mass, and $\ell_i(\boldsymbol{h}_k)$ a neural load head that captures the configuration-dependent gravitational and coupling load. All physical parameters are trained in log-space with hard clamps to physically plausible ranges, so they can adapt to the data without leaving their meaning behind. The trapezoidal position update (5.15) is the same identity (5.10) that the strict data satisfies almost exactly. The neural residuals r_p, r_v, r_x are tanh-bounded to 10 % of the pressure step limit, 10 % of the velocity step limit, and 1 cm respectively: large enough to absorb model error, too small to let the network bypass the physics. This bounded-residual layering is the central robustness argument of the grey-box design — even a poorly trained residual cannot produce an unphysical state jump.

5.4.4 Dataset and training

The command-ROM dataset joins the aligned ID and FD closed-loop exports on exact run identifiers and time indices, yielding the 452 400 transitions and run-level 936/52/52 split summarized in Section 5.1; the logged valve commands enter only as supervision for the internal valve head. Training (seed 17, Adam, learning rate 3×10^{-4}, weight decay 10^{-5}, batch size 2 048, gradient-norm clipping at 5) follows a four-stage rollout curriculum with horizons $H \in \{1, 5, 20, 50\}$ and stage lengths of 30/30/30/40 epochs: each stage optimizes the weighted one-step losses plus a recursive rollout loss at its horizon, with scheduled sampling ramping from fully teacher-forced to 75 % self-fed within each rollout [Bengio 15]. The loss weights (pressure 1.0, velocity 2.0, extension 1.0, rollout 2.0, valve supervision 1.0, residual penalty 0.1, actuation regularization 0.5) penalize the residual heads explicitly so that explanatory burden stays on the physical terms. The logged force channel receives zero supervised weight: it failed a piston-area consistency audit on the current curriculum ($F \neq \Delta p \cdot A$ within tolerance), and supervising a head on a label that violates the model's own structure would corrupt the pressure chain. The

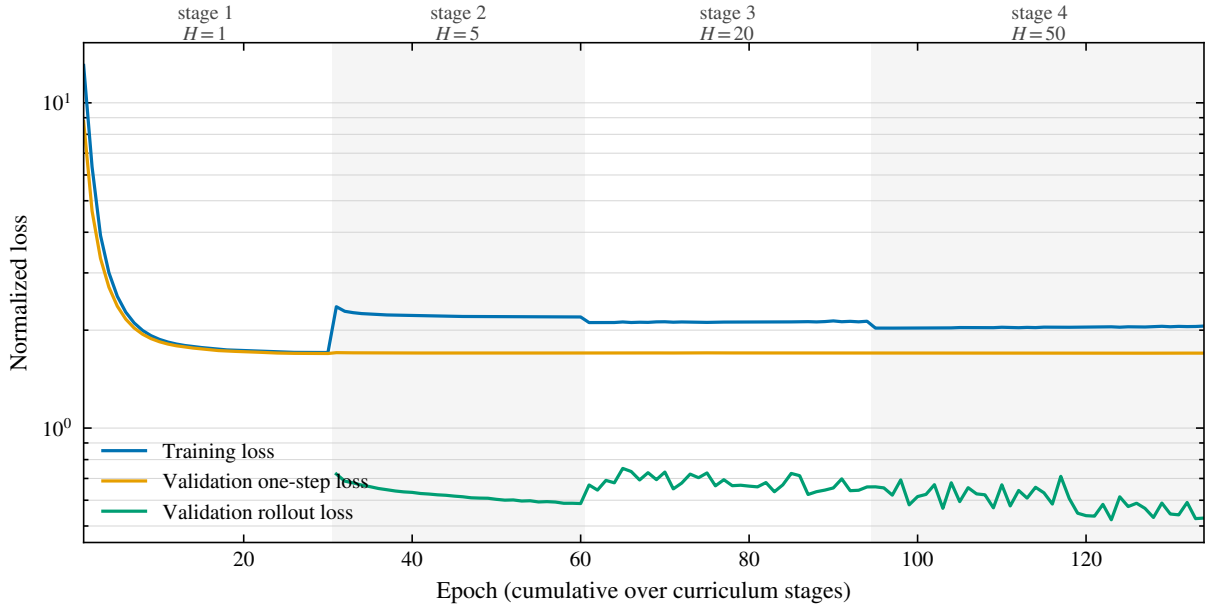


Figure 5.7: Training history of the promoted command-ROM run over the four-stage rollout curriculum (H is the recursive rollout horizon of each stage). The step in training loss at each stage boundary reflects the added longer-horizon rollout term; the validation one-step loss remains stable while the validation rollout loss continues to improve through stage 4.

ROM still exposes its pressure-derived force internally; force supervision can be enabled once a reliable chamber-force label is available.

Figure 5.7 shows the resulting training history. The characteristic step in the training loss at each stage boundary is the newly added, harder rollout term; the important observation is that the validation one-step loss does not degrade as longer horizons are introduced, while the validation rollout loss improves — the curriculum buys long-horizon stability without sacrificing one-step accuracy.

5.4.5 Offline validation

The offline acceptance report (4 096 held-out samples, all gates passed) is summarized in Figure 5.8. Three findings matter for the control experiments. First, *command authority*: with the true command, mean one-step R^2 is 0.998 for the internal valve prediction, 0.64 for the velocity increment, and 0.97 for the extension increment; zeroing or shuffling the command collapses the velocity score to below zero and the extension score to ≈ 0.5, and the 4×4 Jacobian of predicted extension increment with respect to the commanded increment is diagonally dominant with singular values 1.00–0.47 and condition number 2.14. The model responds to what it is asked to do, per axis, which is precisely the property whose absence broke the pretrained FD. Second, *rollout stability*: recursive rollouts remain accurate and bounded to at least 5 s (terminal extension R^2 of 0.998/0.998/0.84/ 0.91 per cylinder), so PPO2 training rollouts do not ride on a diverging model. Third, *honest weaknesses*: the one-step pressure-increment R^2 is low (0.06–0.09) — pressure transients at 0.05 s resolution remain largely unpredictable, and the model's pressure state is best understood as an internal latent that supports the velocity chain rather than as a calibrated pressure estimator — and cylinders 3 and 4 (boom II and head pivot) are consistently

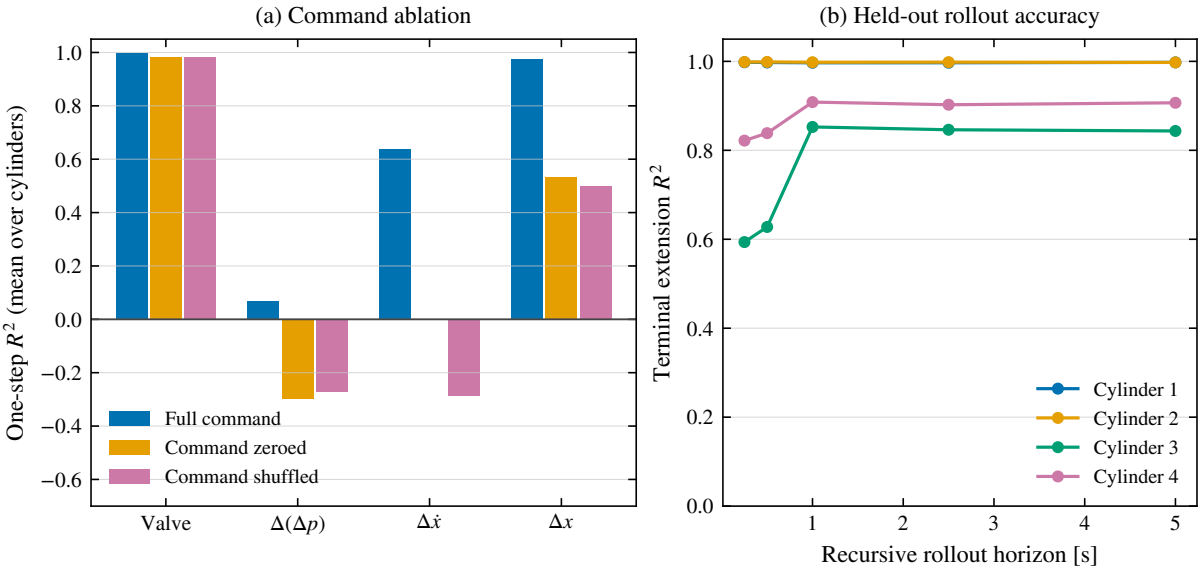


Figure 5.8: Offline validation of the command ROM on held-out runs. (a) One-step R^2 per output group (mean over the four cylinders) with the full desired command, with the command zeroed, and with the command shuffled across the batch; the collapse of the velocity and extension scores without the true command demonstrates command authority. (b) Terminal extension R^2 of recursive rollouts up to 5 s (100 steps) per cylinder.

harder than cylinders 1 and 2 in both velocity and rollout scores. Inference costs 21.5 μs per step in evaluation, which is compatible with the vectorized IsaacLab training loop.

Beyond these aggregate scores, the acceptance protocol requires pressure–velocity phase-plane checks, frequency-response comparisons, and an in-simulator cylinder step gate before any PPO2 training run; a state-only variant trained with the command zeroed throughout serves as the mandatory comparison model for the identifiability ablations in Chapter 7. The nominal, randomized/robust, and latent-conditioned ROM variants used in the PPO2-A/B/C experiments all share the architecture and training protocol of this section and differ only in their training distribution and conditioning inputs; the latent conditioning mechanism is developed in Sections 5.5 and 5.6.

5.5 Latent Representation of Randomized Dynamics

[TODO: write]

5.6 Arm Adaptation Module

[TODO: write]

5.7 Training and Model-Selection Protocol

[TODO: write]

5.8 Offline Model Evaluation

[TODO: write]

6. Control Framework and Simulation Integration

6.1 Nominal Task-Space Controller

[TODO: write]

6.2 Residual PPO Formulation

[TODO: write]

6.3 PPO1-A: Nominal ID-FD Pipeline

[TODO: write]

6.4 PPO1-B: Randomized/Robust ID-FD Pipeline

[TODO: write]

6.5 PPO2-A: Nominal ROM Pipeline

[TODO: write]

6.6 PPO2-B: Randomized/Robust ROM Pipeline

[TODO: write]

6.7 PPO2-C: AAM-Conditioned ROM Pipeline

[TODO: write]

6.8 Optional Oracle-Latent Diagnostic

[TODO: write]

6.9 IsaacLab Integration and Timing

[TODO: write]

6.10 Stability and Constraint Handling

[TODO: write]

7. Experimental Evaluation and Discussion

7.1 Evaluation Questions and Hypotheses

[TODO: write]

7.2 Experimental Matrix and Fairness Controls

[TODO: write]

7.3 Model-Level Experiments

[TODO: write]

7.4 PPO Training Behaviour

[TODO: write]

7.5 Task-Space Trajectory Tracking

[TODO: write]

7.6 Robustness and Generalization Tests

[TODO: write]

7.7 Adaptation Experiments

[TODO: write]

7.8 Ablation Studies

[TODO: write]

7.9 Metrics and Statistical Reporting

[TODO: write]

7.10 Integrated Discussion

[TODO: write]

7.11 Threats to Validity

[TODO: write]

8. Conclusion and Outlook

8.1 Summary of the Approach

[TODO: write]

8.2 Answers to the Research Questions

[TODO: write]

8.3 Main Contributions

[TODO: write]

8.4 Limitations

[TODO: write]

8.5 Future Work

[TODO: write]

A. System Parameters and Coordinate Mappings

[TODO: write]

B. Dataset and Randomization Details

[TODO: write]

C. Model Architectures and Training Settings

[TODO: write]

D. PPO and IsaacLab Configuration

[TODO: write]

E. Supplementary Results

[TODO: write]

Bibliography

[Åström 95] K. J. Åström, B. Wittenmark, *Adaptive Control*, 2nd ed., Reading, MA: Addison-Wesley, 1995.

[Ba 16] J. L. Ba, J. R. Kiros, G. E. Hinton, "Layer Normalization", arXiv preprint arXiv:1607.06450, 2016.

[Bengio 15] S. Bengio, O. Vinyals, N. Jaitly, N. Shazeer, "Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28. 2015, pp. 1171–1179.

[Benner 15] P. Benner, S. Gugercin, K. Willcox, "A Survey of Projection-Based Model Reduction Methods for Parametric Dynamical Systems", *SIAM Review*, vol. 57, no. 4, pp. 483–531, 2015.

[Cho 14] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, Y. Bengio, "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation", in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2014, pp. 1724–1734.

[Chua 18] K. Chua, R. Calandra, R. McAllister, S. Levine, "Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31. 2018.

[Egli 20] P. Egli, M. Hutter, "Towards RL-Based Hydraulic Excavator Automation", in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2020, pp. 2692–2697.

[Egli 22] P. Egli, M. Hutter, "A General Approach for the Automation of Hydraulic Excavator Arms Using Reinforcement Learning", *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 5679–5686, 2022.

[Elfwing 18] S. Elfwing, E. Uchibe, K. Doya, "Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning", *Neural Networks*, vol. 107, pp. 3–11, 2018.

[He 16] K. He, X. Zhang, S. Ren, J. Sun, "Deep Residual Learning for Image Recognition", in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778.

[Hochreiter 97] S. Hochreiter, J. Schmidhuber, "Long Short-Term Memory", *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[Huber 64] P. J. Huber, "Robust Estimation of a Location Parameter", *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964.

[Hwangbo 19] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, M. Hutter, "Learning Agile and Dynamic Motor Skills for Legged Robots", *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.

[Isermann 11] R. Isermann, M. Münchhof, *Identification of Dynamic Systems: An Introduction with Applications*, Berlin, Heidelberg: Springer, 2011.

[Jelali 03] M. Jelali, A. Kroll, *Hydraulic Servo-systems: Modelling, Identification and Control*, London: Springer, 2003.

[Johannink 19] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, S. Levine, "Residual Reinforcement Learning for Robot Control", in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2019, pp. 6023–6029.

[Karniadakis 21] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, L. Yang, "Physics-Informed Machine Learning", *Nature Reviews Physics*, vol. 3, no. 6, pp. 422–440, 2021.

[Kingma 15] D. P. Kingma, J. Ba, "Adam: A Method for Stochastic Optimization", in *International Conference on Learning Representations (ICLR)*. 2015.

[Kumar 21] A. Kumar, Z. Fu, D. Pathak, J. Malik, "RMA: Rapid Motor Adaptation for Legged Robots", in *Proceedings of Robotics: Science and Systems (RSS)*. 2021.

[Lee 20] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, M. Hutter, "Learning Quadrupedal Locomotion over Challenging Terrain", *Science Robotics*, vol. 5, no. 47, p. eabc5986, 2020.

[Ljung 99] L. Ljung, *System Identification: Theory for the User*, 2nd ed., Upper Saddle River, NJ: Prentice Hall, 1999.

[Lutter 19] M. Lutter, C. Ritter, J. Peters, "Deep Lagrangian Networks: Using Physics as Model Prior for Deep Learning", in *International Conference on Learning Representations (ICLR)*. 2019.

[Makoviychuk 21] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, G. State, "Isaac Gym: High Performance GPU-Based Physics Simulation for Robot Learning", arXiv preprint arXiv:2108.10470, 2021.

[Mattila 17] J. Mattila, J. Koivumäki, D. G. Caldwell, C. Semini, "A Survey on Control of Hydraulic Robotic Manipulators With Projection to Future Trends", *IEEE/ASME Transactions on Mechatronics*, vol. 22, no. 2, pp. 669–680, 2017.

[McKay 79] M. D. McKay, R. J. Beckman, W. J. Conover, "A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code", *Technometrics*, vol. 21, no. 2, pp. 239–245, 1979.

[Merritt 67] H. E. Merritt, *Hydraulic Control Systems*, New York: John Wiley & Sons, 1967.

[Mittal 23] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlekar, B. Babich, G. State, M. Hutter, A. Garg, "Orbit: A Unified Simulation Framework for Interactive Robot Learning Environments", *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3740–3747, 2023.

[MULAG Fahrzeugwerk, Heinz Wössner GmbH u. Co. KG 26] MULAG Fahrzeugwerk, Heinz Wössner GmbH u Co KG, "FME 500 Front Mower — Product Documentation", Oppenau, Germany. `https://www.mulag.de/en/roadside-equipment/products/unimog-equipment/front-mower/fme` 2026. Accessed July 2026.

[Nagabandi 18] A. Nagabandi, G. Kahn, R. S. Fearing, S. Levine, "Neural Network Dynamics for Model-Based Deep Reinforcement Learning with Model-Free Fine-Tuning", in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2018, pp. 7559–7566.

[Nakamura 86] Y. Nakamura, H. Hanafusa, "Inverse Kinematic Solutions With Singularity Robustness for Robot Manipulator Control", *ASME Journal of Dynamic Systems, Measurement, and Control*, vol. 108, no. 3, pp. 163–171, 1986.

[Nguyen-Tuong 11] D. Nguyen-Tuong, J. Peters, "Model Learning for Robot Control: A Survey", *Cognitive Processing*, vol. 12, no. 4, pp. 319–340, 2011.

[Paszke 19] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32. 2019, pp. 8024–8035.

[Peng 18] X. B. Peng, M. Andrychowicz, W. Zaremba, P. Abbeel, "Sim-to-Real Transfer of Robotic Control with Dynamics Randomization", in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. 2018, pp. 3803–3810.

[Rudin 22] N. Rudin, D. Hoeller, P. Reist, M. Hutter, "Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning", in *Proceedings of the Conference on Robot Learning (CoRL)*. 2022, pp. 91–100.

[Schoukens 19] J. Schoukens, L. Ljung, "Nonlinear System Identification: A User-Oriented Road Map", *IEEE Control Systems Magazine*, vol. 39, no. 6, pp. 28–99, 2019.

[Schulman 16] J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation", in *International Conference on Learning Representations (ICLR)*. 2016.

[Schulman 17] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, "Proximal Policy Optimization Algorithms", arXiv preprint arXiv:1707.06347, 2017.

[Siciliano 09] B. Siciliano, L. Sciavicco, L. Villani, G. Oriolo, *Robotics: Modelling, Planning and Control*, London: Springer, 2009.

[Silver 18] T. Silver, K. Allen, J. Tenenbaum, L. Kaelbling, "Residual Policy Learning", arXiv preprint arXiv:1812.06298, 2018.

[Slotine 91] J.-J. E. Slotine, W. Li, *Applied Nonlinear Control*, Englewood Cliffs, NJ: Prentice Hall, 1991.

[Srivastava 14] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting", *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[Sutton 18] R. S. Sutton, A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., Cambridge, MA: MIT Press, 2018.

[The MathWorks, Inc. 24] The MathWorks, Inc , "Simscape Fluids: User's Guide", Natick, MA, USA, 2024. `https://www.mathworks.com/products/simscape-fluids.html`.

[Tobin 17] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, P. Abbeel, "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World", in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2017, pp. 23–30.

[von Szadkowski 20] K. von Szadkowski, S. Reichel, "Phobos: A Tool for Creating Complex Robot Models", *Journal of Open Source Software*, vol. 5, no. 45, p. 1326, 2020.

[Wampler 86] C. W. Wampler, "Manipulator Inverse Kinematic Solutions Based on Vector Formulations and Damped Least-Squares Methods", *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 16, no. 1, pp. 93–101, 1986.